Advanced Image Synthesis

Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas

Universidad de Granada

Advanced Image Synthesis

Los Del DGIIM, losdeldgiim.github.io

Arturo Olivares Martos

Granada, 2025-2026

Índice general

1. Graphics Pipeline

During this chapter, we will discuss the graphics pipeline, which is a sequence of steps that a computer graphics system uses to render a 3D scene onto a 2D screen. It starts from the initial creation of 3D models and ends with the final display of the rendered image. There are two types of graphics pipelines: the fixed-function pipeline and the programmable pipeline.

1.1. Fixed-Function Pipeline

The fixed-function pipeline is a traditional graphics pipeline that was widely used in older graphics hardware. It consists of a series of predefined stages that perform specific tasks. Each stage has a fixed function, and the data flows through these stages in a specific order. During the following subsections, we will discuss the stages of the fixed-function pipeline in detail.

1.1.1. Geometry

During this stage, the 3D models are created and defined. This includes defining the vertices, normals, texture coordinates, and other attributes of the 3D objects. In order to create the 3D models, the following primitives are used:

- `GL_POINTS`
- `GL_LINES`: A series of disconnected straight lines.
- `GL_LINE_STRIP`: A series of connected lines.
- `GL_LINE_LOOP`: A series of connected lines that form a loop.
- `GL_POLYGON`: A filled polygon defined by a series of vertices.
- `GL_TRIANGLES`: A series of disconnected triangles.
- `GL_TRIANGLE_STRIP`: A series of connected triangles. The first three vertices define the first triangle, and each subsequent vertex forms a new triangle with the previous two vertices.
- `GL_TRIANGLE_FAN`: A series of connected triangles that share a common central vertex. The first vertex is the center of the fan, and each subsequent vertex forms a new triangle with the center vertex and the previous vertex.
- `GL_QUADS`: A series of disconnected quadrilaterals.

- `GL_QUAD_STRIP`: A series of connected quadrilaterals. The first four vertices define the first quadrilateral, and each subsequent pair of vertices forms a new quadrilateral with the previous two vertices.

Even though all of these primitives are supported by OpenGL, it is recommended to use triangles for rendering, as they are the most efficient and widely supported primitive in modern graphics hardware.

In order to specify the vertices of the primitives, a process evolved from immediate mode to vertex arrays and eventually to vertex buffer objects (VBOs).

1. Immediate Mode: In this mode, vertices are specified one at a time. It is simple to use but inefficient, as it requires multiple function calls for each vertex. The geometry is then stored in the code.

2. Vertex Arrays: This mode allows you to specify an array of vertices and their attributes, which can be more efficient than immediate mode. The geometry is then stored in the RAM.

3. Vertex Buffer Objects (VBOs): This mode allows you to store vertex data in the GPU's memory, which can significantly improve performance.

1.1.2. Geometry Processing

During this stage, the vertices defined in the geometry stage are processed to prepare them for rendering. There are different spaces in which the vertices are processed:

- Object Space: The original space in which the vertices are defined.
- World Space: The space in which the vertices are transformed to position them in the scene.
- Camera/Eye Space: The space in which the vertices are transformed to position them relative to the camera (the camera is at the origin).
- Clip Space: The space in which the vertices are transformed to prepare them for clipping (removing parts of the geometry that are outside the view frustum).
- Normalized Device Space: The space in which the vertices are transformed to fit within a standardized coordinate system (usually a cube from -1 to 1 in all axes).
- Screen Space: The final space in which the vertices are transformed to fit the actual screen dimensions.

In order to transform the vertices from one space to another, several transformations are applied.

Model Transformation

Before moving again to computer graphics, we should focus on the transformations that we can apply to the vertices.

Scaling It is a linear application that changes the size of an object. It can be uniform (same scaling factor for all axes) or non-uniform (different scaling factors for each axis). An scaling transformation of values (α, β, γ) can be represented in 3D using the matrix (1.1).

$$\begin{pmatrix} \alpha & 0 & 0 \\ 0 & \beta & 0 \\ 0 & 0 & \gamma \end{pmatrix} \tag{1.1}$$

The inverse of a scaling transformation is also a scaling transformation with the reciprocal of the scaling factors.

Shearing It is a linear application that distorts the shape of an object by shifting its vertices in a specific direction depending on their distance to a reference plane. A shearing transformation of value α in the x direction depending on their distance to the XZ plane can be represented in 3D using the matrix (1.2).

$$\begin{pmatrix} 1 & \alpha & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{1.2}$$

Rotation It is a linear application that rotates an object around a specific axis by a certain angle. A rotation transformation of angle θ around the z axis can be represented in 3D using the matrix (1.3).

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{1.3}$$

The inverse of a rotation transformation is also a rotation transformation with the opposite angle. Mathematically, given that the rotation matrix is orthogonal, its inverse is equal to its transpose.

Translation This last type of transformation is not a linear application, as it does not preserve the origin. These transformations need to be represented using homogeneous coordinates, which add an extra dimension to the vertices. Therefore:

- Points are represented as $(x, y, z, 1)^t$.
- Vectors are represented as $(x, y, z, 0)^t$.

Every transformation that we have allready seen can be represented in homogeneous coordinates by adding an extra row and column to the transformation matrix, with 0s in the new row and column, except for the last element of the

new column, which is 1. For example, a translation transformation of values (α, β, γ) can be represented in 3D using the matrix (1.4).

$$\begin{pmatrix} 1 & 0 & 0 & \alpha \\ 0 & 1 & 0 & \beta \\ 0 & 0 & 1 & \gamma \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{1.4}$$

The model transformation is the transformation that takes the vertices from object space to world space. It is used to position the objects in the scene. It is usually represented as a combination of different transformations, and mathematically it can be represented as a single matrix M_{model} that is the product of the individual transformation matrices (taking the order of transformations into account).

$$P_{\text{world}} = M_{\text{model}} \cdot P_{\text{object}}$$

When a transformation M is applied to a vertex P, the normal vector N of the vertex is also transformed. Someone might think that the normal vector can be transformed using the same transformation M, but this is not always the case. The normal vector should be transformed using the inverse transpose of the transformation matrix M to ensure that it remains perpendicular to the surface after the transformation. Mathematically, the transformed normal vector N' can be calculated as follows:

$$N' = (M^{-1})^t \cdot N$$

Demostración. To understand why the normal vector should be transformed using the inverse transpose of the transformation matrix, we need to consider how transformations affect the geometry of the surface. Let n be the normal vector of a surface at a point P and t be the tangent vector at the same point. For clarity, let's use T_O to denote the transformation applied to the points (the tangent) and T_N to denote the transformation applied to the normal vector. We want to ensure that after applying the transformations, the normal vector remains perpendicular to the tangent vector, which means:

$$(T_N \cdot n) \perp (T_O \cdot t) \iff (T_N \cdot n) \cdot (T_O \cdot t) = 0 \iff (T_N \cdot n)^t \cdot (T_O \cdot t) = 0 \iff n^t \cdot T_N^t \cdot T_O \cdot t = 0$$

Given that n is perpendicular to t, we have $n^t \cdot t = 0$. Therefore, for the transformed normal vector to remain perpendicular to the transformed tangent vector, we need:

$$T_N^t \cdot T_O = I \iff T_N^t = T_O^{-1} \iff T_N = (T_O^{-1})^t$$

□

This should always be taken into account when applying transforming normals, *the normal vector should be transformed using the inverse transpose of the transformation matrix applied to the points.*

View Transformation

The view transformation is the transformation that takes the vertices from world space to camera/eye space. It is used to position the camera in the scene and to define the view direction. In OpenGL, the camera is always positioned at the origin of the coordinate system, looking in the z direction. As the camera can't move, the view transformation is achieved by applying the inverse of the camera's transformation to the vertices.

Let's say that we want to position the camera at a point C in world space, looking at a direction D. We can define the view transformation using the following steps:

1. First of all, as we want to position the camera at point C, we need to apply a translation transformation that moves the world in the opposite direction of C. This can be represented using a translation matrix T that translates by $-C$.

2. Next, we need to align the camera's view direction with the z axis. To do this, we only know the direction D that the camera is looking at.

   a) We can calculate the right vector R of the camera using the cross product of the view direction D and the up vector U' (which is usually defined as $(0, 1, 0)$). This can be represented as $R = D \times U$.

   b) Then, we can calculate the up vector U of the camera using the cross product of the right vector R and the view direction D. This can be represented as $U = R \times D$.

   This way, the matrix $[R, U, D]$ maps this orthogonal basis to the standard basis of the camera space. Therefore, the inverse of this transformation is:

$$[R, U, D]^{-1}$$

Therefore, the view transformation can be represented as the product of the translation and the inverse of the rotation:

$$M_{\text{view}} = [R, U, D]^{-1} \cdot T$$

Lighting and Shading

Before moving to the next stage, lighting and shading should be considered, because they are the processes that determine the color and intensity of the pixels in the final rendered image. Lighting is the process of calculating how light interacts with the surfaces of the objects in the scene, while shading is the process of determining the color of each pixel based on the lighting calculations. The Phong Lighting Model is a widely used model for calculating the lighting in computer graphics. It uses the following notation.

Notación. The notation used in the Phong Lighting Model is shwon in Figure 1.1.

- X is the point on the surface being shaded.

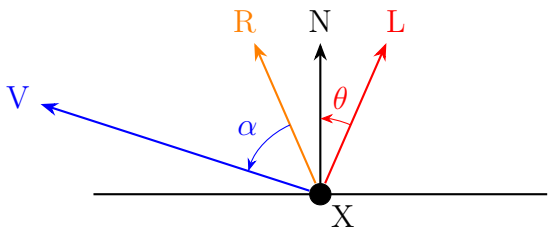

Figura 1.1: Notation used in the Phong Lighting Model.

- N is the normal unitary vector at point X.
- L is the unitary vector from point X to the light source.
- V is the unitary vector from point X to the viewer (camera).
- R is the reflection of the light vector L around the normal vector N, calculated as $R = 2(N \cdot L)N - L$.

  This calculation is valid because $N \cdot L = \cos(\theta)$ gives the projection of L onto N (as the cosine is the adjacent side of the triangle formed by L and N).

The Phong Lighting Model calculates the color of a pixel using three components: ambient, diffuse, and specular.

- Ambient Component: This component represents the constant background light in the scene. It is calculated as:

  $$\text{Ambient}(X) = C_{X,a} \cdot C_{L,a}$$

  where $C_{X,a}$ is the ambient color of the surface at point X and $C_{L,a}$ is the ambient color of the light source.

- Diffuse Component: This component represents the light that is scattered in all directions when it hits a surface. It is calculated as:

  $$\text{Diffuse}(L, X) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, \cos(\theta)) = C_{X,d} \cdot C_{L,d} \cdot \text{máx}(0, N \cdot L)$$

  where $C_{X,d}$ is the diffuse color of the surface at point X, $C_{L,d}$ is the diffuse color of the light source, and θ is the angle between the normal vector N and the light vector L.

- Specular Component: This component represents the light that is reflected in a specific direction when it hits a surface. It is calculated as:

  $$\text{Specular}(L, X, V) = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, \cos(\alpha))^n = C_{X,s} \cdot C_{L,s} \cdot \text{máx}(0, R \cdot V)^n$$

  where $C_{X,s}$ is the specular color of the surface at point X, $C_{L,s}$ is the specular color of the light source, α is the angle between the reflection vector R and the view vector V, and n is the shininess coefficient that controls the size of the specular highlight.

The final color of the pixel is then calculated as the sum of these three components:

$$\text{Color}(L, X, V) = \text{Ambient}(X) + \text{Diffuse}(L, X) + \text{Specular}(L, X, V)$$

Perspective Transform

In this stage, the vertices are transformed from camera/eye space to clip space. The next definition is important.

Definición 1.1 (Frustrum)**.** The view frustum is a geometric shape that represents the volume of space that is visible to the camera. It can be defined in several ways (using the field of view, just the planes...). Two important planes are:

- Near Plane: The plane that is closest to the camera. Objects that are closer than this plane will not be rendered.
- Far Plane: The plane that is farthest from the camera. Objects that are farther than this plane will not be rendered.

Two type of frustrums can be defined:

- Perspective Frustum: A frustum that represents a perspective projection, where objects that are farther from the camera appear smaller. This type of frustum looks like a truncated pyramid, with the near plane being smaller than the far plane.
- Orthographic Frustum: A frustum that represents an orthographic projection, where objects that are farther from the camera appear the same size as objects that are closer to the camera. This type of frustum looks like a rectangular box, with the near and far planes being the same size. This type of projection is often used for 2D rendering or for technical drawings where accurate measurements are important. It represents that the viewer is at the infinity.

The perspective transformation is then used to transform the vertices from camera/eye space to clip space. It is represented as a matrix that prepares the vertices for perspective division. The exact form of the perspective transformation matrix depends on the parameters of the view frustum (how was it given), but it generally includes terms that account for the field of view, aspect ratio, and the near and far planes. We will simply consider the $M_{\text{projection}}$ matrix as the one that transforms the vertices from camera/eye space to clip space, without going into the details of its construction. Therefore, what the programmer should implement in OpenGL is:

$$P_{\text{clip}} = M_{\text{projection}} \cdot M_{\text{view}} \cdot M_{\text{model}} \cdot P_{\text{object}}$$

Clipping

The clipping stage removes any vertices that are outside the view frustum. Here, the vertices are transformed from clip space to normalized device space by performing perspective division. Given that the vertices are in homogeneous coordinates and we need the last w coordinate to be 1, the other coordinates are divided by w to achieve this. After perspective division, and given that the correct perspective transformation was applied, the vertices will be in normalized device space, and only the vertices that are within the range of $[-1, 1]$ in all axes will be kept for further processing. The vertices that are outside this range will be clipped, meaning that they will be discarded and not rendered.

Viewport Transformation

In this last stage, the vertices are transformed from normalized device space to screen space. This transformation maps the normalized device coordinates to the actual pixel coordinates on the screen. The viewport transformation is defined by the viewport parameters, which specify the position and size of the viewport on the screen. The viewport transformation can be represented as a matrix that scales and translates the vertices to fit within the specified viewport. After this transformation, the vertices are ready to be rasterized and rendered on the screen.

1.1.3. Rasterization

The rasterization stage is the process of converting the vertices and primitives into pixels on the screen. During this stage, the graphics pipeline determines which pixels on the screen correspond to each primitive, and a fragment is generated for each of these pixels. The data for each vertex (color, texture coordinates, etc.) is interpolated across the primitive to determine the attributes of each fragment.

Data Interpolation

Given $n+1$ points $\overrightarrow{x}_i \in \mathbb{R}^k$ with their corresponding values $f_i \in \mathbb{R}$, $i \in \{0, \ldots, n\}$, interpolating is finding a function $f : \mathbb{R}^k \to \mathbb{R}$ such that $f(\overrightarrow{x}_i) = f_i$ for all $i \in \{0, \ldots, n\}$. When achieved, the function f can be used to estimate the value of f at any point $\overrightarrow{x} \in \mathbb{R}^k$ by evaluating $f(\overrightarrow{x})$.

Linear interpolation is a simple method of interpolation that only considers the linear functions f, which can be represented as $f(\overrightarrow{x}) = \overrightarrow{a} \cdot \overrightarrow{x} + b$ for some $\overrightarrow{a} \in \mathbb{R}^k$ and $b \in \mathbb{R}$. Given that we have $n+1$ constraints (one for each point) and $k+1$ unknowns (the components of $\overrightarrow{a}$ and b), we can solve for the coefficients of the linear function f if $n+1 \leqslant k+1$. In our case, we are interested in interpolating the attributes of the vertices across the primitives, which means that we have 3 vertices (for triangles) and two dimensions (the screen coordinates), so $k = 2$. Therefore, our aim is to find $a, b, c \in \mathbb{R}$ such that $f(x, y) = ax + by + c$ and $f(\overrightarrow{x}_i) = f_i$ for $i \in \{0, 1, 2\}$. This can be achieved by solving the system of equations of Equation (1.5).

$$\begin{cases} ax_0 + by_0 + c = f_0 \\ ax_1 + by_1 + c = f_1 \\ ax_2 + by_2 + c = f_2 \end{cases} \implies \begin{pmatrix} x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \end{pmatrix} \tag{1.5}$$

However, this does not take the perspective into account. To achieve perspective-correct interpolation, we need to consider the depth of each vertex.

Shading

Shading is the process of determining the color of each pixel based on the lighting calculations. During rasterization, the attributes of the vertices are interpolated across the primitive to determine the attributes of each fragment. There are different ways to apply the Phong Lighting Mode, depending on what is interpolated and when is the color calculated:

- Flat Shading: The color is calculated for each primitive (e.g., triangle) using a single value of each attribute for the whole primitive. This results in a faceted appearance, as each primitive has a single color.
- Gouraud Shading: The color is calculated at each vertex , and then interpolated across the primitive. This results in a smoother appearance than flat shading, but it can produce artifacts such as Mach bands.
- Phong Shading: The normal vector is interpolated across the primitive, and the color is calculated for each fragment using the interpolated normal vector. This results in a smoother appearance than Gouraud shading and can produce more accurate lighting effects.

1.1.4. Fragment Processing

The fragment processing stage is almost the final stage of the graphics pipeline, where the fragments generated during rasterization are processed to determine their final color and whether they should be rendered on the screen. During this stage, several operations can be performed on the fragments, such as depth testing, stencil testing, blending, and more. The specific operations performed during fragment processing depend on the settings and configurations of the graphics pipeline.

Textures

A texture map is a discretely sampled multi-dimensional function containing material properties, like color or normals. Texture mapping is the process of applying a texture map to a 3D model to add detail and realism to the rendered image. During fragment processing, the texture coordinates of each fragment are used to sample the corresponding attribute from the texture map, which can then be used. Some concepts about textures are:

- Wrapping: Texture coordinates usually are planned to be in the range of $[0, 1]$, but sometimes they can be outside this range. Wrapping is the process of determining how to handle texture coordinates that are outside the $[0, 1]$ range. Common wrapping modes include:
  - `GL_REPEAT`: The texture is repeated when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_MIRRORED_REPEAT`: The texture is repeated and mirrored when the texture coordinates are outside the $[0, 1]$ range.
  - `GL_CLAMP_TO_EDGE`: The texture coordinates are clamped to the edge of the texture when they are outside the $[0, 1]$ range. This means that the texture will be stretched to fill the area outside the $[0, 1]$ range.

  One mode should be selected for each texture coordinate (e.g., s and t).
- Filtering: Usually, the texture maps are of a different resolution than the rendered image, which means that the texture needs to be filtered to determine the color of each pixel. There are two types of filtering:

- `Minification`: The process of determining the color of a fragment when the texels (texture pixels) are smaller than the screen pixels.
- `Magnification`: The process of determining the color of a fragment when the texels are larger than the screen pixels.

Common filtering modes include:

- `GL_NEAREST`: The color of the fragment is determined by the color of the nearest texel. In minification, this can lead to aliasing artifacts, while in magnification, it can lead to a blocky appearance.
- `GL_LINEAR`: The color of the fragment is determined by the bilinear interpolation of the colors of the 4 nearest texels. In minification it does not make a big difference (considering 4 texels when the pixel may be covered by 100 texels does not improve the quality), but in magnification it can improve the appearance of the texture, making it appear smoother.

  In order to achieve bilinear interpolation, it is simply interpolated in both dimensions.

- MIP Mapping: MIP mapping is a technique used to improve the quality of textures when they are minificated. It involves creating multiple levels of detail for a texture, where each level is a downscaled version of the original texture. During rendering, the appropriate level of detail is selected based on the distance and angle of the fragment being rendered. This can help to reduce aliasing artifacts and improve the overall appearance of the texture, as no minification is applied when the appropriate level of detail is selected.

Fragment Tests

After the fragment processing stage, several tests can be performed on the fragments to determine whether they should be rendered on the screen or discarded. Before performing these tests, the content of each pixel should be explained. Each pixel (not fragment) on the screen has a buffer that stores:

- Color: The color of the pixel, usually represented as RGBA (red, green, blue, alpha).
- Depth: The depth value of the current pixel, which is the depth value of the fragment that is currently rendered at that pixel (the closest fragment to the camera).
- Stencil: The stencil value of the pixel, which is used for stencil testing. The programmer can define the stencil value for each pixel and store it in the stencil buffer.

Once this has been explained, some common fragment tests include:

- Alpha Test: This test discards fragments based on their alpha value. It consists of a comparison between the alpha value of the fragment and a reference value, using a specified comparison function (e.g., less than, greater than, equal to). If the comparison fails, the fragment is discarded and not rendered on the screen.

- Stencil Test: This test discards fragments based on their stencil value. It compares the stencil value of the pixel where the fragment would be rendered with a reference value, using a specified comparison function. If the comparison fails, the fragment is discarded and not rendered on the screen. The stencil test can be used for various effects, such as creating shadows, outlining objects, or implementing complex masking operations.
- Depth Test: This test discards fragments based on their depth value. It compares the depth value of the fragment with the depth value of the corresponding pixel in the depth buffer. If the fragment is closer to the camera than the existing pixel, it passes the depth test and is rendered on the screen, and the depth buffer is updated with the new depth value. If the fragment is farther from the camera than the existing pixel, it fails the depth test and is discarded.

Blending

Blending is the process of combining the color of a fragment with the color of the corresponding pixel in the framebuffer to produce a final color that is rendered on the screen. Blending is typically (but not only) used to achieve transparency effects, where the color of the fragment is blended with the color of the background to create a semi-transparent appearance. The blending operation is defined by:

$$C_{\text{final}} = C_{\text{source}} \cdot F_{\text{source}} \odot C_{\text{destination}} \cdot F_{\text{destination}}$$

where:

- C_{final} is the final color that is rendered on the screen.
- C_{source} is the color of the fragment being processed.
- $C_{\text{destination}}$ is the color of the corresponding pixel in the framebuffer.
- F_{source} and $F_{\text{destination}}$ are the blending factors for the source and destination colors, respectively. These factors determine how much of each color contributes to the final color. Common blending factors include:
  - `GL_ZERO`: The factor is 0, meaning that the corresponding color does not contribute to the final color.
  - `GL_ONE`: The factor is 1, meaning that the corresponding color fully contributes to the final color.
  - `GL_SRC_ALPHA`: The factor is equal to the alpha value of the source color, meaning that the contribution of the source color is proportional to its transparency.
  - `GL_ONE_MINUS_SRC_ALPHA`: The factor is equal to 1 minus the alpha value of the source color, meaning that the contribution of the destination color is proportional to the transparency of the source color.

  The specific blending factors used can be configured based on the desired effect.
- $\odot$ represents the operator used to combine the source and destination colors. Common operators include addition, subtraction, minimum, and maximum.

1.1.5. Framebuffer

The framebuffer is the final destination for the rendered image. It is a memory buffer that stores the color, depth, and stencil information for each pixel on the screen. After all the fragment processing and tests have been performed, the final color of each pixel is written to the framebuffer, which is then displayed on the screen. The framebuffer can also be used for off-screen rendering, where the rendered image is stored in a texture or a renderbuffer instead of being displayed on the screen. This allows for various effects and techniques, such as post-processing, shadow mapping, and more.

1.2. Programmable Pipeline

The programmable pipeline is a modern graphics pipeline that allows developers to write custom shaders to control the behavior of each stage of the pipeline. This provides greater flexibility and control over the rendering process, allowing for more complex and realistic graphics. The programmable pipeline consists of several stages, including vertex shading, tessellation, geometry shading, fragment shading, and more. Each stage can be programmed using a shading language such as GLSL (OpenGL Shading Language) or HLSL (High-Level Shading Language). The programmable pipeline has largely replaced the fixed-function pipeline in modern graphics hardware, as it allows for more advanced rendering techniques and greater performance.

2. Reflection

During this chapter, we will discuss how can reflections be simulated in computer graphics. We will start with the simplest case of planar reflections, and then we will move to more complex cases of non-planar reflections and refraction.

2.1. Planar Reflection

A planar reflection is a reflection that occurs on a flat surface, such as a mirror or a calm body of water. Let's first approach it from a mathematical perspective, and then we will see how it can be implemented in computer graphics. Let p be a point in a surface with normal vector N, and let q be the point that we want to reflect over the surface, obtaining the reflected point q_m. Let α be the angle between the vector $\overrightarrow{pq}$ and the normal vector N. Our aim is to find the coordinates of q_m given the coordinates of p, q, and N. The situation is illustrated in the Figure 2.1.

Given that the surface is planar, we can q_m as:

$$q_m = q - 2N \cdot \text{distance}(q, \text{plane})$$

In order to find that distance, we have:

$$\left.\begin{aligned} N \cdot \overrightarrow{pq} = \|N\| \cdot \|\overrightarrow{pq}\| \cdot \cos(\alpha) = 1 \cdot \|\overrightarrow{pq}\| \cdot \cos(\alpha) \\ \cos(\alpha) = \frac{\text{adjacent}}{\text{hypotenuse}} = \frac{\text{distance}(q, \text{plane})}{\|\overrightarrow{pq}\|} \end{aligned}\right\} \Longrightarrow$$

$$\Longrightarrow \text{distance}(q, \text{plane}) = \cos(\alpha) \cdot \|\overrightarrow{pq}\| = N \cdot \overrightarrow{pq}$$

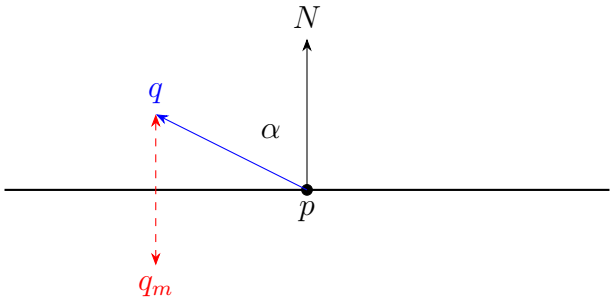

Figura 2.1: Planar reflection.

Therefore, we can express q_m as:

$$\begin{aligned} q_m &= q - 2N(N \cdot \overrightarrow{pq}) \\ &= q - 2N(N \cdot (q - p)) \\ &= q - 2N(N_x(q_x - p_x) + N_y(q_y - p_y) + N_z(q_z - p_z)) \end{aligned}$$

We should take into account that this formula is valid in $3D$. Therefore, the coordinates of q_m can be expressed as:

$$\begin{aligned} q_{m_x} &= q_x - 2N_x(N_x(q_x - p_x) + N_y(q_y - p_y) + N_z(q_z - p_z)) \\ &= (1 - 2N_x^2)q_x + (-2N_xN_y)q_y + (-2N_xN_z)q_z + 2N_x(N_xp_x + N_yp_y + N_zp_z) \\ q_{m_y} &= q_y - 2N_y(N_x(q_x - p_x) + N_y(q_y - p_y) + N_z(q_z - p_z)) \\ &= (-2N_yN_x)q_x + (1 - 2N_y^2)q_y + (-2N_yN_z)q_z + 2N_y(N_xp_x + N_yp_y + N_zp_z) \\ q_{m_z} &= q_z - 2N_z(N_x(q_x - p_x) + N_y(q_y - p_y) + N_z(q_z - p_z)) \\ &= (-2N_zN_x)q_x + (-2N_zN_y)q_y + (1 - 2N_z^2)q_z + 2N_z(N_xp_x + N_yp_y + N_zp_z) \end{aligned}$$

Using $k = N \cdot p$ (treating p as a vector), we can express q_m as:

$$\begin{aligned} q_{m_x} &= (1 - 2N_x^2)q_x + (-2N_xN_y)q_y + (-2N_xN_z)q_z + 2N_xk \\ q_{m_y} &= (-2N_yN_x)q_x + (1 - 2N_y^2)q_y + (-2N_yN_z)q_z + 2N_yk \\ q_{m_z} &= (-2N_zN_x)q_x + (-2N_zN_y)q_y + (1 - 2N_z^2)q_z + 2N_zk \end{aligned}$$

Therefore, we have the following matrix representation of the planar reflection:

$$\begin{pmatrix} q_{m_x} \\ q_{m_y} \\ q_{m_z} \\ 1 \end{pmatrix} = \begin{pmatrix} 1 - 2N_x^2 & -2N_xN_y & -2N_xN_z & 2N_xk \\ -2N_yN_x & 1 - 2N_y^2 & -2N_yN_z & 2N_yk \\ -2N_zN_x & -2N_zN_y & 1 - 2N_z^2 & 2N_zk \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix}$$

This matrix can be used to perform the planar reflection in computer graphics. We can apply this transformation to the vertices of the objects that we want to reflect, and then we can render the reflected objects as usual.

One first approach to implement planar reflections in computer graphics would be:

1. Render the reflected objects using the planar reflection transformation.
2. Disable color buffer writing, so that the mirror surface is not rendered (just the depth buffer is updated).
3. Render the mirror surface.
4. Enable color buffer writing.
5. Render the original objects.

However, this approach has some issues, such as the fact that the reflected objects are not clipped by the mirror surface. The stencil buffer can be used to solve this issue, by marking the pixels that belong to the mirror surface and then using this information to clip the reflected objects. In addition, a clip plane is used to avoid rendering the parts of the reflected objects that are in front of the mirror surface.

2.1.1. Recursive Reflections

Recursive reflections occur when an object is reflected in a mirror, and then this reflection is reflected again in another mirror, and so on. This can lead to an infinite number of reflections, which can be computationally expensive to render. One way to handle recursive reflections is to limit the number of reflections that are rendered, which will reduce the computational cost while still providing a visually appealing result. However, this approach can lead to artifacts, such as the fact that the reflections will not be accurate after a certain number of reflections. This leads to the need of finding alternative solutions to handle recursive reflections which can provide a more accurate result while still being computationally efficient.

Alternative Solution

An alternative solution to handle planar reflections is to use a technique called "render-to-texture". This technique involves reflecting the camera across the mirror surface (not only the position, but also the direction), and then rendering the reflected scene from there. This rendered image can then be used as a texture to apply to the mirror surface, which will give the illusion of a reflection. This technique is more efficient than rendering every reflection recursively, as it only requires rendering the scene once.

2.2. Non-Planar Reflection

Non-planar reflections occur when the reflecting surface is not flat, such as a curved mirror or a body of water with waves. In this case, the reflection is more complex to calculate, as the normal vector at each point of the surface can vary. One way to handle non-planar reflections is to use a technique called "environment mapping", which uses "cube maps" to store the reflection information for each direction.

A cube map is a texture that consists of six square images, each representing the reflection in a different direction (positive and negative x, y, and z). In order to generate the cube map, the camera is placed at the center of the reflecting surface, and the scene is rendered six times, each time with a different view direction corresponding to one of the faces of the cube map. Once the cube map is generated, it can be used to sample the reflection information for each pixel on the reflecting surface. The cube map has two important properties:

- It is view-independent, meaning that the reflection information is the same regardless of the position of the viewer. It only has to be updated when the reflecting surface moves or the scene changes.
- It is an approximation of the reflection, as it is only phisically perfect at the center of the reflecting surface.

The texture coordinates for sampling the cube map are however easy to calculate.

1. First of all, the reflexion vector is calculated. Let p be the point on the reflecting surface whose reflection we want to calculate, and let N be the normal vector

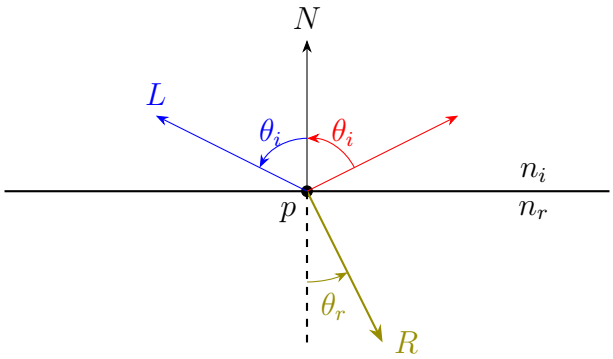

Figura 2.2: Refraction of light.

at point p. Let V be the vector from point p to the viewer (camera). The reflection vector R can be calculated as:

$$R = 2(N \cdot V)N - V$$

2. Once we have the reflection vector R, the largest absolute component of R represents the direction of the reflection. For example, if $|R_x|$ is the largest component, then the reflection is in the x direction. Then, the sign of that component determines whether the reflection is in the positive or negative direction. For example, if R_x is positive, then the reflection is in the positive x direction, and if R_x is negative, then the reflection is in the negative x direction. This information allows us to determine which face of the cube map we need to sample from.

3. Finally, the other two components of the reflection vector R are divided by the largest component to obtain the 2D texture coordinates for sampling the cube map in a range of $[-1, 1]$.

2.3. Refraction

The refraction of light occurs when light passes from one medium to another, such as from air to water. This causes the light to bend, which can create interesting visual effects. The amount of bending depends on the angle of incidence and the refractive indices of the two media. The refractive index is a measure of how much a material slows down light compared to vacuum. The relationship between the angle of incidence, the angle of refraction, and the refractive indices is given by Snell's Law:

$$n_i \sin(\theta_i) = n_r \sin(\theta_r)$$

where n_i and n_r are the refractive indices of the first and second media, respectively, and θ_i and θ_r are the angles of incidence and refraction, respectively. The situation is illustrated in the Figure 2.2.

2.4. Fresnel Effects

The Fresnel effects describes the attenuation of reflected light depending on the angle of incidence, the frequency of the light, and the material properties. The equations for calculating the Fresnel effects are difficult to solve, but they are usually approximated.

3. Transparency

During this chapter, we will discuss the concept of transparency in the context of Image Synthesis. Transparency is a crucial aspect of computer graphics that allows for the creation of images with varying levels of opacity. It enables the blending of different layers and the creation of complex visual effects.

Usually, we render with the depth test enabled, resulting in a final image where only the closest fragments are visible. In order to allow transparency, this test is disabled and the fragments are blended together. The blending process is controlled by the blending function and the blending factors, as explained in Section 1.1.4.

- The *Blending Function* is the addition.
- The source blending factor is the alpha value of the fragment, and the destination blending factor is one minus the alpha value of the fragment. This allows for the creation of semi-transparent effects, where the color of the fragment is blended with the color of the background based on the alpha value of the fragment in front of it.

Therefore, the blending operation can be expressed as in Equation 3.1, and the code needed is shown in Listing 1.

$$C_{\text{final}} = \alpha_{\text{source}} \cdot C_{\text{source}} + (1 - \alpha_{\text{source}}) \cdot C_{\text{destination}} \tag{3.1}$$

However, this has a big problem: *blending is not commutative*. This means that the order in which the fragments are blended together matters, and it can lead to incorrect results if the fragments are not blended in the correct order. To solve this problem, we need to sort the fragments based on their depth values before blending them together. Therefore, the general approach is:

1. Render all opaque objects first, with the depth test enabled and blending disabled.

```
1 glEnable(GL_BLEND);
  glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
  glBlendEquation(GL_FUNC_ADD);
```

Código fuente 1: Code for enabling blending with the specified blending function and factors.

2. Disable the test buffer updates, but enable the depth test and blending.

3. Render all transparent objects, sorted from farthest to nearest, with blending enabled.

However, this approach has a big problem: the sorting step can be computationally expensive (or even not possible) in scenes with a large number of transparent objects or with complex geometry. For instance, in a situation where three transparent objects are intersecting each other, it may not be possible to determine a clear sorting order for the fragments. In such cases, alternative techniques such as depth peeling can be used to achieve correct blending results without the need for sorting.

3.1. Depth Peeling

Depth peeling is a technique used to achieve correct blending results for transparent objects without the need for sorting. It works by rendering the scene multiple times, each time peeling away the closest layer of fragments. This allows for the correct blending of fragments based on their depth values, even in cases where sorting is not possible. The DP algorithm uses two depth buffers: one for storing the depth values of the closest fragments (reference), and another for storing the depth values of the next closest fragments (the one used in each layer). The algorithm can be summarized as follows:

1. In the first pass, render the scene with the depth test enabled and blending disabled, and store the depth values of the closest fragments in the reference depth buffer.

   This first layer of fragments is stored.

2. In the following passes, the depth buffer from the previous pass is used to peel away fragments with depth values less than or equal to the stored depth values. This allows to ignore the fragments that have already been rendered in the previous passes.

   All other fragments are rendered using the working depth buffer, and the depth values of the next closest fragments are stored in the reference depth buffer.

   Each layer of fragments is also stored.

This process is repeated until all layers of fragments have been rendered and blended together. The final result is a correctly blended image of the transparent objects, without the need for sorting. However, obtaining the Depth Complexity (number of layers) is not trivial.

3.1.1. Depth Complexity

There are several techniques to obtain the depth complexity of a scene, which is the number of layers that need to be rendered in the depth peeling algorithm. Some of these techniques include:

- The "logic" approach. Rendering the scene with the depth test disabled and incrementing the stencil buffer for each fragment. The maximum value in the stencil buffer will give the depth complexity of the scene.

  In order to make it more efficient, the maximum value in the stencil buffer can be obtained using a reduction operation, which is a parallel algorithm that obtains the maximum value in a buffer by recursively dividing the buffer into smaller sub-buffers and comparing the values in each sub-buffer to find the maximum value.

- Using *Occlusion Queries*. This technique involves using occlusion queries to ask the GPU how many fragments were rasterized into pixels in the last `draw` call.

  We just use the algorithm described in the previous section, but each time we ask the GPU if any fragments were rasterized in the last pass. If no fragments were rasterized, we can stop the algorithm, as we have already rendered all the layers of fragments in the scene.

3.1.2. Abuse of the MSAA

An important drawback of the depth peeling algorithm is that it requires multiple rendering passes, which can be computationally expensive. However, the MSAA can be abused to achieve a the same effect requiring much less rendering passes.

Multisample Anti-Aliasing (MSAA) is a technique used to improve the quality of rendered images by reducing aliasing artifacts. It works by sampling multiple points within each pixel and averaging the results to produce a smoother final image. However, MSAA can also be used to extract multiple layers of fragments in a single rendering pass. Instead of saving multiple samples per pixel, we can save multiple layers of fragments in the samples.

The stencil buffer is used to control which layer of fragments is being stored in each sample. We set the stencil buffer associated with the sample 0 to 2, the stencil buffer associated with the sample 1 to 3, and so on. Then, for each layer of fragments, it is written to the sample whose stencil buffer value is 2, and then the stencil buffer value is decremented by 1 for every sample.

The initial stencil buffer values start from 2 in order to control that no more layers are stored than the number of samples available. When all the samples have been used, the stencil buffer values will 0 for all the examples but the last one, which will have a stencil buffer value of 1. If all the values are 0, it means that an overflow has occurred.

Lastly, if more layers are needed than the number of samples available, multiple rendering passes are required. The initial stencil buffer values can be set to $kn + 2$, where k is the number of samples available and n is the number of rendering passes already performed. This way, we can skip the first kn layers of fragments, which have already been rendered in the previous passes, and start rendering the next layers

of fragments directly in the samples. This is limited by the maximum value of the stencil buffer, which is 255 in most implementations, so the maximum number of layers that can be rendered using this technique is 255.

4. Shadows

Shadows are an essential aspect of computer graphics that contribute to the realism and depth of a scene. They provide visual cues about the spatial relationships between objects and the light sources in the scene. In this chapter, we will discuss various techniques for generating shadows in computer graphics.

4.1. Projective Shadows

This technique involves projecting the geometry of the shadow-casting object onto a plane that represents the surface on which the shadow will be cast. Let's fist approach this technique from a mathematical perspective. Consider a point light source located at position l, a point q whose shadow we want to calculate, and a plane defined by a point p and a normal vector N. This situation is illustrated in Figure 4.1. The goal is to find the point q_s on the plane where the shadow of point q will be cast.

Given the normal vector $N = (A, B, C)$ of the plane and the point p on the plane, we can derive the equation of the plane as follows:

$$Ax + By + Cz + D = 0, \qquad D = -(Ap_x + Bp_y + Cp_z).$$

The shadow point q_s can be expressed, for certain $t \in \mathbb{R}$, as:

$$q_s = q + t \cdot (\overrightarrow{lq}) = q + t \cdot (q - l)$$

One approach could be to substitute q_s into the plane equation and solve for t. However, a more efficient method is to make firstly an assumption about the position of the light source.

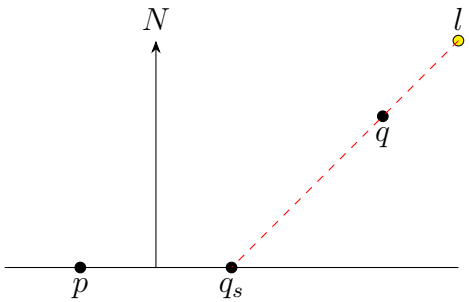

Figura 4.1: Projective shadow calculation.

- Let's firstly suppose that the light source is located at the origin, i.e., $l = (0, 0, 0)$. In this case, the shadow point can be expressed as $q_s = t \cdot q$. Substituting this into the plane equation gives us:

$$A(tq_x) + B(tq_y) + C(tq_z) + D = 0 \implies t = -\frac{D}{Aq_x + Bq_y + Cq_z} = -\frac{D}{\overrightarrow{N} \cdot q}$$

  Therefore, the shadow point can be calculated as:

$$q_s = -\frac{D}{\overrightarrow{N} \cdot q} \cdot q$$

  Given that we work with homogeneous coordinates and that the perspective division will be applied, we have that the shadow point can be calculated as:

$$\begin{pmatrix} -D & 0 & 0 & 0 \\ 0 & -D & 0 & 0 \\ 0 & 0 & -D & 0 \\ A & B & C & 0 \end{pmatrix} \cdot \begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \equiv \begin{pmatrix} q_{s_x} \\ q_{s_y} \\ q_{s_z} \\ 1 \end{pmatrix}$$

- Now, let's consider the general case where the light source is located at an arbitrary position $l = (l_x, l_y, l_z)$. In this case, let T be the translation matrix for the $\overrightarrow{l}$ vector. Therefore, the shadow point can be calculated as:

$$T \cdot \begin{pmatrix} -D & 0 & 0 & 0 \\ 0 & -D & 0 & 0 \\ 0 & 0 & -D & 0 \\ A & B & C & 0 \end{pmatrix} \cdot T^{-1} \cdot \begin{pmatrix} q_x \\ q_y \\ q_z \\ 1 \end{pmatrix} \equiv \begin{pmatrix} q_{s_x} \\ q_{s_y} \\ q_{s_z} \\ 1 \end{pmatrix}$$

In both cases, we obtain a matrix that can be used to project the shadow of any point q onto the plane defined by p and N. This is then used in the rendering pipeline to create the visual effect of shadows in the scene. After rendering the scene, we can render it again with the shadow projection matrix applied to the geometry of the shadow-casting objects, and the resulting colors will be blended with the background to create the final shadow effect.

Lastly, a bias can be applied to the shadow projection to prevent z-fighting artifacts, which occur when the shadow geometry has the same depth as the surface it is projected onto, causing flickering or other visual artifacts. This bias can be implemented by slightly offsetting the shadow geometry closer to the camera, ensuring that it is rendered in front of the surface and thus avoiding z-fighting.

This technique is relatively simple and efficient, but it has some limitations, because it only works well for flat surfaces and can produce artifacts when the shadow-casting object is close to the surface. Additionally, it does not account for self-shadowing or shadows cast by other objects in the scene.

4.2. Shadow Volumes

Shadow volumes are a more advanced technique for generating shadows that can handle complex scenes with multiple light sources and self-shadowing. The idea is to create a volume that represents the region of space that is in shadow, and then use this volume to determine which parts of the scene are in shadow.

4.2.1. Volumes Generation

The first step in the shadow volume technique is to generate the shadow volume for each possible shadow-casting object in the scene. This is done by extruding the silhouette of the object away from the light source to create a volume that represents the region of space that is in shadow. This is done in three steps:

- Possible silhouette edges are extruded to infinity away from the light source, creating the walls of the shadow volume.
- The back-facing occlder's triangles (with respect to the light source) are extruded to infinity, creating the back cap of the shadow volume.
- The front-facing occlder's triangles (with respect to the light source) are "slightly" extruded away from the light source, creating the front cap of the shadow volume.

This process results in a closed volume that represents the region of space that is in shadow. Once the shadow volumes are generated, they can be used in the rendering pipeline to determine which parts of the scene are in shadow.

4.2.2. Rendering with Shadow Volumes

Here, there are two main approaches to render the scene with shadow volumes: the z-pass and z-fail methods. Both methods involve using the stencil buffer to keep track of which pixels are in shadow.

The z-pass method

First of all, the unlit scene is rendered. Then, for each light source, the following steps are performed:

1. The stencil buffer is cleared to zero, color write and depth write are disabled, but the depth test is enabled. It means that it will only update the stencil buffer.
2. Render the front-facing polygons of the shadow volume (with respect to the camera). For each fragment that *passes* the depth test, the stencil buffer of the corresponding pixel is incremented by one. It represents the fact that the ray from the camera to the pixel has entered the shadow volume.

   The stencil buffer is only updated when the depth test passes, which means that only the fragments that are not ocluded by other real geometry of the scene will affect.

3. Render the back-facing polygons of the shadow volume (with respect to the camera). For each fragment that *passes* the depth test, the stencil buffer of the corresponding pixel is decremented by one. It represents the fact that the ray from the camera to the pixel has exited the shadow volume.

4. Finally, render the lit scene, but only for the pixels where the stencil buffer is equal to zero.

    - If it was greater than zero, it means that the ray from the camera to the pixel is still inside a shadow volume, and thus the pixel is in shadow.
    - If it was less that zero it would mean that the ray from the camera to the pixel has exited more shadow volumes than it has entered, which would mean that the camera is inside a shadow volume. However, this would simply not happen because of overflow issues of the stencil buffer, which is typically implemented as an unsigned integer.

As I mentioned before, the z-pass method can produce artifacts when the camera is inside a shadow volume, because the stencil buffer will not be updated correctly. This happens because the near plane clips the shadow volume, causing the front-facing polygons to be clipped and thus not updating the stencil buffer when they should. To solve this issue, the z-fail method can be used.

The z-fail method

The z-fail method is similar to the z-pass method, but instead of counting from the camera to the real-scene geometry, it counts from the infinity to the real-scene geometry. As it happened with the z-pass method, the unlit scene is rendered first. Then, for each light source, the following steps are performed:

1. The stencil buffer is cleared to zero, color write and depth write are disabled, but the depth test is enabled.

2. Render first the back-facing polygons of the shadow volume (with respect to the camera). For each fragment that *fails* the depth test, the stencil buffer of the corresponding pixel is incremented by one. It represents the fact that the ray, after the real-scene geometry, has exited the shadow volume.

3. Render then the front-facing polygons of the shadow volume (with respect to the camera). For each fragment that *fails* the depth test, the stencil buffer of the corresponding pixel is decremented by one. It represents the fact that the ray, after the real-scene geometry, has entered the shadow volume.

4. Render the lit scene, but only for the pixels where the stencil buffer is equal to zero.

    - If it was greater than zero, it means that the ray, after the real-scene geometry, has exited more shadow volumes than it has entered. It means that the real-scene geometry was in shadow, and thus the pixel is in shadow.

- If it was less that zero it would mean that the ray, after the real-scene geometry, has entered more shadow volumes than it has exited. It would mean that the shadow volumes are not closed (because perspective lets you moving the far plane to infinity), which would be a bug in the shadow volume generation process. In addition, this would not be possible because of overflow issues of the stencil buffer, which is typically implemented as an unsigned integer.

As we can see, the z-fail method solves the problems that the z-pass method had, but this technique is still computationally expensive, because it requires rendering the shadow volume geometry multiple times. In addition, volumes have to be regenerated every frame if the light source or the shadow-casting objects are moving, which can further increase the computational cost.

4.3. Shadow Mapping

Shadow mapping is the most widely used technique for generating shadows in real-time applications, such as video games. The idea is to render the scene from the perspective of the light source to create a depth map, which is then used to determine which parts of the scene are in shadow when rendering from the camera's perspective. The process of shadow mapping can be broken down into two main steps: creating the shadow map and rendering the scene with shadows.

4.3.1. Creating the Shadow Map

The first step in shadow mapping is to create the shadow map. This is done by rendering the scene from the perspective of the light source, using a depth buffer to store the distance from the light source to the closest surface at each pixel. The resulting depth map is called the shadow map. The shadow map is typically stored as a texture, where each texel contains the depth value corresponding to the distance from the light source to the closest surface in that direction. The steps in detail to create the shadow map are as follows:

1. Render the scene from the perspective of the light source.

   First of all, the coordinates of the vertices of the scene are transformed to world space (using the inverse of the matrixes needed). Once in world space, we have to multiply the vertices by the following matrixes:

   - Instead of the usual "view matrix" that transforms the vertices from world space to camera space, we use a "light view matrix" that transforms the vertices from world space to light space.
   - A projection matrix that transforms the vertices from light space also has to be used.

   At this point, the scene is rendered from the perspective of the light source, and the depth values are then stored in the depth buffer.

2. The depth buffer is then saved as a texture, which will be used as the shadow map in the next step.

   Before doing so, and given that the coordinates in the clip space are in the range $[-1, 1]$, we have to transform them to the range $[0, 1]$ to be able to store them in a texture. This can be done by multiplying all of the coordinates by $1/2$ and then adding $1/2$ to all of the coordinates. Once done so, the depth buffer can be saved as a texture, which will be used as the shadow map in the next step.

4.3.2. Rendering the Scene with Shadows

The second step in shadow mapping is to render the scene from the camera's perspective, using the shadow map to determine which parts of the scene are in shadow. When rendering the scene as usual, the following steps are performed for each fragment:

1. The world coordinates of the fragment are transformed to light space using the same light view and projection matrixes that were used to create the shadow map. Also the same transformation to the range $[0, 1]$ is applied to the coordinates. This gives us the corresponding texture coordinates in the shadow map.

2. The depth value stored in the shadow map at the corresponding texture coordinates is retrieved. This depth value represents the distance from the light source to the closest surface in that direction.

3. The depth value of the fragment being rendered is compared to the depth value retrieved from the shadow map. Two things may happen:

   - If the depth value of the fragment is greater than the depth value retrieved from the shadow map, it means that the light does not reach the fragment, and thus the fragment is in shadow. In this case, the fragment is shaded accordingly.
   - If the depth value of the fragment is less than or equal to the depth value retrieved from the shadow map, it means that the light reaches the fragment, and thus the fragment is not in shadow. In this case, the fragment is shaded as usual.

In this case, a bias can be applied to the depth value of the fragment being rendered to prevent shadow acne, which is a common artifact in shadow mapping that occurs when the depth values of the fragments being rendered are very close to the depth values stored in the shadow map, causing weird patterns of shadow to appear on the surfaces. This bias can be implemented by adding a small constant value to the depth value of the fragment being rendered, ensuring that it is slightly further away from the light source than the depth values stored in the shadow map, thus preventing shadow acne.

Resolution of the Shadow Map

One of the main issues with shadow mapping is that the resolution of the shadow map can affect the quality of the shadows. If the shadow map has a low resolution, it can result in blocky or pixelated shadows. The quality is specially significant in shadows closer to the camera, because just one texel covers a larger area of the scene and much more fragments will be affected by the same depth value stored in the shadow map. To mitigate this issue, some techniques can be used, such as:

- Increasing the resolution of the shadow map, which can improve the quality of the shadows, but it also increases the computational cost and memory usage.
- Cascaded shadow maps

  This technique involves using multiple shadow maps with different resolutions for different parts of the scene. For example, a high-resolution shadow map can be used for the area close to the camera, while a lower-resolution shadow map can be used for the area further away from the camera. This allows for better quality shadows in the areas where it is most noticeable, while still keeping the computational cost and memory usage manageable.
- Trapezoidal shadow maps

  This idea is based on the observation that the area of the scene that is visible from the light source is often not a square, but rather a trapezoid. By using a trapezoidal shadow map instead of a square one, we can better fit the shadow map to the visible area of the scene, which can improve the quality of the shadows while still keeping the resolution manageable.

It should be noted that all the shadows we have created are hard shadows, which means that they have a sharp edge. In reality, shadows often have soft edges due to the size of the light source and the distance from the shadow-casting object to the surface on which the shadow is cast. To create soft shadows, additional techniques can be used, but they are beyond the scope of this chapter. Physics Based Rendering (PBR) techniques can also be used to create more realistic shadows by simulating the physical properties of light and materials. This will be covered in more detail in next chapters.

5. Radiance Transfer

During this chapter, we will introduce the two differet types of illumination, local and indirect illumination, to end up developping the global illumination. In order to explain these concepts, we first have to introduce some radiometric quantities. Radiometry is the science of measuring electromagnetic radiation, including visible light. It provides a set of quantities that are used to describe the properties of light and how it interacts with surfaces.

Definición 5.1 (Radiant Energy)**.** Radiant energy is the total amount of energy that is carried by a beam of light.

$$Q \qquad [J]$$

Definición 5.2 (Radiant Flux)**.** Also known as Radiant Power, it is the rate at which radiant energy is transferred or emitted. It is defined as the amount of radiant energy per unit time.

$$\Phi = \frac{dQ}{dt} \qquad [W = {}^{J}/_{s}]$$

Definición 5.3 (Irradiance)**.** Irradiance is the amount of radiant flux that is incident on a surface per unit area.

$$E = \frac{d\Phi}{dA} \qquad [{}^{W}/_{m^2}]$$

For the next measures, the solid angle is needed. It is a measure of the amount of space that an object occupies in the field of view of an observer. It is defined as the area of the projection of the object onto a unit sphere centered at the observer. More generally, if the sphere used for the projection has a radius r, and the surface area of the projection is A, the solid angle Ω is defined as:

$$\Omega = \frac{A}{r^2} \qquad [sr]$$

The unit of solid angle is the steradian (sr) (an adimensional unit), which is defined as the solid angle subtended by an area of r^2 on the surface of a sphere of radius r. It is the three-dimensional equivalent of the two-dimensional radian.

Definición 5.4 (Radiant Intensity)**.** Radiant intensity is the amount of radiant flux that is emitted by a source in a particular direction per unit solid angle.

$$I = \frac{d\Phi}{d\vec{\omega}} \qquad [{}^{W}/_{sr}]$$

Definición 5.5 (Radiance)**.** Radiance is the amount of radiant flux that is emitted, reflected, transmitted or received by a surface per projected unit area and per unit solid angle.

$$L = \frac{d^2\Phi}{dA \cdot \cos(\theta) \cdot d\overrightarrow{\omega}} \qquad [W/m^2 \cdot sr]$$

where θ is the angle between the normal of the surface and the direction of the incoming light. The term $\cos(\theta)$ is used to account for the fact that the effective area of the surface that is illuminated by the light decreases as the angle of incidence increases.

Radiance is the most used quantity in computer graphics, because it is the one that is conserved[1] along a ray of light. This means that the radiance of a ray of light does not change as it travels through space, unless it interacts with a surface. This property makes it easier to compute the illumination of a scene, because we can trace rays of light from the camera to the scene and compute the radiance at each point of intersection.

Observación. So many measures may be confusing. The basic one is the radiant energy, and then the radiant flux is the rate of change of the radiant energy. All of the other measures are derived from the flux energy connecting it with other quantities:

- Irradiance: Flux per unit area.
- Radiant Intensity: Flux per unit solid angle.
- Radiance: Flux per unit area and per unit solid angle.

5.1. Local Illumination

The local illumination model is the simplest one, and it only takes into account the direct light that hits a surface. It does not consider any light that is reflected or refracted by other surfaces. It is explained in the Equation 5.1.

$$L_O = \rho \cdot L_I \cdot \cos(\theta) = \rho \cdot L_I \cdot N \cdot L \tag{5.1}$$

Where L_O is the outgoing radiance, L_i is the incoming radiance, θ is the angle between the normal of the surface and the direction of the incoming light, N is the normal of the surface and L is the direction of the incoming light. The term ρ is the reflectance of the surface, which is a measure of how much light is reflected by the surface. It is a value between 0 and 1.

5.2. Indirect Illumination

Light is not only emitted by light sources as the local illumination model assumes, but it can also be reflected or refracted by other surfaces. This aspects are the ones addressed by the indirect illumination model, which is more accurate than the local

[1] In fact, it only happens in vacuum.

illumination model, but also more computationally expensive. When an observer looks at a point on a surface, three types of rays should be taken into account to determine the illumination of that point:

- Shadow ray: it is traced from the point of intersection to the light source, and it is used to determine if the point is in shadow or not.
- Reflection ray: if the surface is reflective, a reflection ray is traced in the direction of the reflected light, and it is used to determine which object is visible in the reflection.
- Refraction ray: if the surface is transparent, a refraction ray is traced in the direction of the refracted light, and it is used to determine which object is visible through the transparent surface.

5.3. Global Illumination

The global illumination model unifies the local and indirect illumination models, and it represents how light really is. It is usually called Physically Based Rendering (PBR), because it takes into account all the light that is emitted, reflected, transmitted or received by a surface. It is the most accurate model, but it is also the most computationally expensive. It is based on the Rendering Equation 5.2, which is an integral equation that describes the equilibrium of light in a scene.

$$L_O(x, \omega, \lambda) = L_E(x, \omega, \lambda) + \int_\Omega f_r(x, \omega, \omega', \lambda) \cdot L_I(x, \omega', \lambda) \cdot (\omega' \cdot N)\ d\omega' \qquad (5.2)$$

where $L_O(x, \omega, \lambda)$ is the outgoing radiance at point x in direction ω and wavelength λ, L_E is the emitted radiance, f_r is the BRDF, L_I is the incoming radiance, N is the normal of the surface and Ω is the hemisphere of directions above the surface. The integral term represents the contribution of all the incoming light that is reflected by the surface, and it is computed by integrating over all the directions in the hemisphere.

Observación. It should be noted that máx$(0, \omega' \cdot N)$ is not needed, as the domain of integration is the hemisphere of directions above the surface, which means that $\omega' \cdot N$ is always non-negative. However, if the domain of integration were the entire sphere, then the term máx$(0, \omega' \cdot N)$ would be needed to ensure that only the directions above the surface contribute to the integral.

5.3.1. Bidirectional Reflectance Distribution Function (BRDF)

The BRDF is a function that describes how light is reflected by a surface. It is defined as the fraction of light from an incoming direction ω' that is reflected in an outgoing direction ω at a point x on the surface, for a given wavelength λ. It is denoted as $f_r(x, \omega, \omega', \lambda)$ and it has the following properties:

- $f_r(x, \omega, \omega', \lambda) \geqslant 0$: the BRDF is always non-negative.

- Helmholtz Reciprocity, which states that the BRDF is symmetric with respect to the incoming and outgoing directions. This means that the light may be inverted.

$$f_r(x, \omega, \omega', \lambda) = f_r(x, \omega', \omega, \lambda)$$

- Energy Conservation, which states that the total amount of light reflected by a surface cannot exceed the amount of light that is incident on it.

$$\int_\Omega f_r(x, \omega, \omega', \lambda) \cdot (\omega' \cdot N)\ d\omega' \leqslant 1 \qquad \forall x, \omega, \lambda$$

5.3.2. Algorithms for Global Illumination

As the user may expect, actually computing the global illumination is a very complex task, because it requires solving the rendering equation, which is an integral equation. There are several algorithms that, taking the rendering equation as a base, try to compute the global illumination in a more efficient way. In the next sections, we will introduce some of the most popular algorithms for global illumination.

Recursive Ray Tracing

Recursive ray tracing is a simple algorithm that traces rays of light from the camera to the scene. For each object that is intersected by a ray, two new rays are traced:

- Reflection ray: if the surface is reflective, a reflection ray is traced in the direction of the reflected light.
- Refraction ray: if the surface is transparent, a refraction ray is traced in the direction of the refracted light.

The algorithm continues tracing rays until:

- A ray does not intersect any object
- A ray intersects a light source
- A ray reaches a maximum recursion depth
- A ray contributes less than a certain threshold to the final image

This algorithm, which may appear that does not produce good results, is actually the basis of many other algorithms for global illumination.

Path Tracing

As recursive ray tracing, path tracing is an algorithm that traces rays of light from the camera to the scene. However, for each object that is intersected by a ray, only one new ray is traced, which is chosen randomly according to the BRDF of the surface. As happened with recursive ray tracing, the algorithm continues tracing rays until:

- A ray does not intersect any object
- A ray intersects a light source
- A ray reaches a maximum recursion depth
- A ray contributes less than a certain threshold to the final image

Path tracing is a more efficient algorithm than recursive ray tracing, and it may produce better results, because it takes into account the BRDF of the surface. However, it is still a very computationally expensive algorithm because it requires tracing a large number of rays to produce a good image.

6. Precomputed Radiance Transfer (PRT)

In the previous chapter, we introduced the Rendering Equation (5.2) but realized that it is almost impossible to solve in real-time. Some approaches to simulate the idea that that equation introduced were discussed, as Recursive Ray Tracing and Path Tracing. However, these methods are computationally expensive and do not actually solve the equation. In this chapter, we will introduce a method that allows us to solve the Rendering Equation in real-time, but with some limitations. This method is called Precomputed Radiance Transfer (PRT). The main idea behind PRT is to precompute as much of the lighting information as possible, so that at runtime we can quickly evaluate the lighting for a given scene.

6.1. Assumptions and Simplifications

To make the problem of solving the Rendering Equation tractable, we need to make some assumptions and simplifications. The three main simplifications we will make are:

- Lambertian reflector

  A lambertian reflector is a surface that reflects light equally in all directions. This means that the BRDF for a Lambertian reflector is constant and does not depend on the incoming or outgoing directions. This simplification allows us to ignore the directional dependence of the BRDF, which makes the problem easier to solve. It is defined by the material's albedo ρ, which is the fraction of incoming light that is reflected by the surface. The BRDF for a Lambertian reflector can be expressed as:

  $$f_r(x, \omega, \omega', \lambda) = \frac{\rho}{\pi}$$

  where the factor of π is included to ensure energy conservation, as the reflected light is distributed uniformly over the hemisphere.

- Only first van Neumann Term

  The Rendering Equation itself hides recursive terms, as the incoming radiance L_i can be expressed is itself the outgoing radiance from another point in the scene. This means that to solve the equation, we would need to evaluate it recursively. Each level in the recursion is called a van Neumann term, and

represents the contribution of light that has bounced a certain number of times in the scene. The first van Neumann term represents the direct lighting, which is the contribution of light that comes directly from the light sources to the point being shaded. By only considering the first van Neumann term, we are ignoring any indirect lighting, which is the contribution of light that has bounced off other surfaces in the scene before reaching the point being shaded.

- Distant Lighting

  Distant lighting is a simplification that assumes that the light sources in the scene are infinitely far away. This means that the incoming light can be treated as parallel rays, and the direction of the incoming light does not change across the scene.

  This simplification, along with the previous one, allows us to simplify the incoming radiance term in the Rendering Equation as:

  $$L_i(x, \omega', \lambda) = L_{\text{env}}(\omega') \cdot V(x, \omega')$$

  where $L_{\text{env}}(\omega')$ is the radiance coming from the environment in the direction ω', and $V(x, \omega')$ is a visibility term represents whether the point x is visible from the direction ω'.

Therefore, with these simplifications, the Rendering Equation can be rewritten as the Equation (6.1):

$$L_o(x, \omega, \lambda) = L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot V(x, \omega') \cdot (\omega' \cdot N) \; d\omega' \tag{6.1}$$

The term $V(x, \omega') \cdot (\omega' \cdot N)$ is called the transfer function, and it represents how much of the incoming light from the environment is transferred to the outgoing radiance at the point x in the direction ω.

$$T(x, \omega') = V(x, \omega') \cdot \text{máx}(0, \omega' \cdot N)$$

6.2. Converting the Integral into a Dot Product

The idea that PRT introduces is project the possible integrals onto a basis, so that we can simply convert the integral into a dot product. Let's consider the space of all the possible functions with domain A and range B. Given that this space is infinite-dimensional, every basis will have an infinite number of elements. Chosen a basis $\{b_i\}$, we can express any function $f : A \to B$ as a linear combination of the basis elements:

$$f(x) = \sum_{i=0}^{\infty} c_i b_i(x)$$

where c_i are the coefficients of the linear combination:

$$c_i = \langle f(x), b_i(x) \rangle = \int_A f(x) b_i(x) \; dx$$

For the PRT to really simplify the problem, we need to choose an specific type of basis, called an orthogonal basis.

Definición 6.1 (Orthogonal Basis)**.** A basis $\{b_i\}$ is orthogonal if for any two different elements b_i and b_j, the inner product $\langle b_i, b_j \rangle$ is zero. This means:

$$\langle b_i(x), b_j(x) \rangle = \int_A b_i(x) b_j(x)\ dx = \delta_{ij} = \begin{cases} 0 & \text{if } i \neq j \\ 1 & \text{if } i = j \end{cases}$$

In order to understand the idea that lies behind PRT, let's consider two functions f and g, and an orthogonal basis $\{b_i\}$. Given that we are only interested in approximating the integral of the product of f and g, only the first N coefficients of the linear combination will be relevant. Let's then assume that the projection of f, g onto the basis $\{b_i\}$ is given by:

$$f(x) \approx \sum_{i=0}^{N} c_i b_i(x) \qquad g(x) \approx \sum_{i=0}^{N} d_i b_i(x)$$

Then, the integral of the product of f and g can be approximated as:

$$\int_A f(x)g(x)\ dx \approx \int_A \left(\sum_{i=0}^{N} c_i b_i(x) \right) \left(\sum_{j=0}^{N} d_j b_j(x) \right)\ dx = \sum_{i=0}^{N} \sum_{j=0}^{N} c_i d_j \int_A b_i(x) b_j(x)\ dx \overset{(*)}{=}$$
$$\overset{(*)}{=} \sum_{i=0}^{N} c_i d_i \int_A b_i(x) b_i(x)\ dx = \sum_{i=0}^{N} c_i d_i$$

where in $(*)$ we used the fact that the basis is orthogonal. Therefore, we have approximated the integral of the product of f and g as a dot product of their coefficients in the ortogonal basis.

6.2.1. Spherical harmonics (SH)

The choice of the basis is crucial for the success of PRT. The most common choice for the basis in PRT is the Spherical Harmonics (SH). Spherical Harmonics are a set of *orthogonal* functions defined on the surface of a unit sphere and with possible complex values. However, and given that we are only using real functions, we will only consider the real spherical harmonics. The fact that they are defined on the surface of a unit sphere makes them ideal for representing functions that depend on directions, using just *two coordinates* (the azimuth θ and the inclination ϕ; the radius is fixed to 1). Spherical Harmonics are a family with two parameters:

- Band: $l \in \mathbb{N}$
- Mode: $m \in \mathbb{Z} \cap [-l, l]$

Therefore, they are usually referred to as $Y_l^m(\theta, \phi)$, where θ is the azimuth and ϕ is the inclination. The number of coefficients needed to represent a function using Spherical Harmonics up to band L is given by:

$$N = \sum_{l=0}^{L} (2l + 1) = L + 1 + 2 \cdot \frac{L(L+1)}{2} = L + 1 + L(L+1) = (L+1)^2$$

Observación. It should not be confused how many bands are going to be used with up to which band we are going to use.

- Up to band L: we are going to use all the bands from 0 to L, which means that we are going to use $L+1$ bands, and therefore $(L+1)^2$ coefficients.
- L bands: we are going to use the bands from 0 to $L-1$, which means that we are going to use L bands, and therefore L^2 coefficients.

6.2.2. Monte Carlo Integration

The coefficients of the projection of a function f onto the Spherical Harmonics basis have to be computed using an integral over the unit sphere. To compute these integrals, Monte Carlo Integration is usually used.

Monte Carlo Integration is a numerical method for approximating integrals using random sampling. The idea is to sample points from the domain of the function and use the average value of the function at those points to approximate the integral. The more samples we take, the better the approximation will be.

- Monte Carlo Integration is particularly useful for high-dimensional integrals, where traditional numerical methods may struggle.
- It is an abstraction of the Riemann sum, where instead of using a regular grid of points, we use random samples.

Therefore, to compute the coefficients of the projection of a function f onto the Spherical Harmonics basis, we can use Monte Carlo Integration and just sample the function at random points on the unit sphere.

6.3. PRT Pipeline

The PRT pipeline consists of two main stages: the precomputation stage and the rendering stage. The idea is that as much as possible should be precomputed in the first stage, so that the rendering stage can be as fast as possible.

6.3.1. Precomputation Stage

In order to be able to solve the Rendering Equation in real-time, we need to precompute the following information (assuming that we use L bands for approximating):

1. The values of the L^2 SH in each direction. These are no integrals, but they are needed to project the other functions.
2. The projection of the term $L_{\text{env}}(\omega')$ onto the SH basis, which gives us L^2 coefficients for each color channel (R, G, B). For each of the sample directions for the Monte Carlo Integration, we need:
    - The value of the environment map in that direction, which can be obtained by sampling the environment map.

- The value of the SH basis functions in that direction, which can be obtained from the precomputed values of the SH basis functions.

3. For each vertex x in the scene, the projection of the transfer function $T(x, \omega')$ onto the SH basis, which gives us L^2 coefficients for each vertex. For each of the sample directions for the Monte Carlo Integration, we need to compute the following terms:

   - The visibility term $V(x, \omega')$ can be computed using ray tracing, which is a computationally expensive process. However, given that this is done in the precomputation stage, it is not a problem. Depending how far the ray collides, this term will have a higher or lower value.
   - The term $(\omega' \cdot N)$ can be computed using the normal of the vertex and the direction of the incoming light.
   - The value of the SH basis functions in that direction, which can be obtained from the precomputed values of the SH basis functions.

This way, we have precomputed all the information needed to evaluate the Rendering Equation in real-time. The precomputation stage can be computationally expensive, especially for the third step, where we need to compute the visibility term for each vertex and each sample direction. However, this is done offline, so it does not affect the real-time performance of the rendering stage. In this point, we have:

$$L_{\text{env}}(\omega') \approx \sum_{i=0}^{(L-1)^2} l_i b_i(\omega') \qquad T(x, \omega') \approx \sum_{i=0}^{(L-1)^2} t_{x,i} b_i(\omega')$$

6.3.2. Rendering Stage

In the rendering stage, we can now evaluate the Rendering Equation in real-time using the precomputed information. Given a point x and a direction ω, we can compute the outgoing radiance as:

$$\begin{aligned}
L_o(x, \omega, \lambda) &= L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot V(x, \omega') \cdot (\omega' \cdot N)\ d\omega' = \\
&= L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \int_\Omega L_{\text{env}}(\omega') \cdot T(x, \omega')\ d\omega' \approx \\
&\approx L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \int_\Omega \left(\sum_{i=0}^{(L-1)^2} l_i b_i(\omega') \right) \left(\sum_{j=0}^{(L-1)^2} t_{x,j} b_j(\omega') \right)\ d\omega' \overset{(*)}{=} \\
&\overset{(*)}{=} L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \sum_{i=0}^{(L-1)^2} l_i t_{x,i} \int_\Omega b_i(\omega') b_i(\omega')\ d\omega' = \\
&= L_e(x, \omega, \lambda) + \frac{\rho}{\pi} \sum_{i=0}^{(L-1)^2} l_i t_{x,i}
\end{aligned}$$

where in $(*)$ we used the fact that the basis is orthogonal. Therefore, we have approximated the integral of the product of L_{env} and T as a dot product of their

coefficients in the SH basis. This allows us to evaluate the Rendering Equation in real-time, as we only need to compute a dot product of two vectors of size L^2 for each color channel.

One important thing to note is that the SH are rotationally invariant. That means that if we rotate the scene, the coefficients of the projection of the functions onto the SH basis will change, but the values of the functions themselves will not change. This allows us to, instead of recomputing all the coefficients when the scene is rotated, we can just rotate the coefficients of the projection of the functions onto the SH basis. This is done using a rotation matrix. Therefore, we can achieve real-time rendering of the scene even when the scene is rotated, without having to recompute all the coefficients. However, PRT can't be used when the objects in the scene are moving, as the visibility term will change, and therefore we would need to recompute all the coefficients for each vertex, which is not feasible in real-time. Therefore, PRT is only suitable for *static scenes*, where the objects are not moving.

7. Ambient Occlusion

Ambient occlusion is a shading method that calculates how much ambient light can reach a point on a surface. It is used to add depth and realism to 3D scenes by simulating the soft shadows that occur in areas where light is partially blocked by nearby geometry. However, it should be noted that ambient occlusion is not a shadowing technique in the traditional sense, as it does not account for direct light sources or cast shadows. Instead, it focuses on the occlusion of ambient light, which is the diffuse light that is scattered in all directions.

The idea that lays behind ambient occlusion is that points on a surface that are close to other geometry will receive less ambient light and therefore appear darker. This is because the nearby geometry blocks some of the ambient light from reaching those points. Everything is calculated based on the Ambient Occlusion Factor of Equation (7.1), which is a value between 0 and 1 that represents the amount of ambient light that can reach a point on a surface. A value of 0 means that the point is completely occluded and receives no ambient light, while a value of 1 means that the point is fully exposed and receives all ambient light.

$$k_a(p) = \frac{1}{\pi} \int_{\Omega^+} V(\overrightarrow{\omega}_i) \cos\theta_i d\overrightarrow{\omega}_i \tag{7.1}$$

where:

- Ω^+ is the normal-oriented hemisphere above the point p.
- $V(\overrightarrow{\omega}_i)$ is the visibility function, which is 1 if the direction $\overrightarrow{\omega}_i$ is not occluded by any geometry and 0 if it is occluded. This is calculated by casting a ray from the point p in the direction $\overrightarrow{\omega}_i$ and checking for intersections with other geometry in the scene.
- $\cos\theta_i$ is the cosine of the angle between the normal at point p and the direction $\overrightarrow{\omega}_i$. This term accounts for the fact that light arriving at a grazing angle contributes less to the ambient illumination than light arriving directly.
- The factor of $1/\pi$ is used to normalize the result, because the integral of the cosine term over the hemisphere is equal to π; this ensures that the ambient occlusion factor is properly scaled to the range $[0, 1]$.

Therefore, AO is described as follows: "Trace rays through the normal-oriented unit hemisphere, and return the cosine-weighted percentage of rays that do not hit any geometry." This method is computationally expensive, as it requires casting many rays for each point on the surface to accurately estimate the ambient occlusion factor. However, there are various techniques to approximate ambient occlusion.

7.1. Raytracing AO

The idea behind raytracing AO is to preprocess everything for the static geometries in the scene, and store the results in a occlusion map, which essentially is a texture. This method is very efficient for static objects, as the ambient occlusion can be calculated once and reused for each frame. However, it does not help with dynamic objects.

7.2. Object Space AO

Object space AO is a method that calculates ambient occlusion in the object space of a 3D model. This technique represents each vertex of the model as a circle with the same normal as the vectex and with an area equal to one third of the the sum of the areas of the triangles that share the vertex[1]. This representation can be fastly calculated, as the lenght of the sides of the triangles are just an square root, and then there are just some simple calculations to find the area of the triangle known the length of its sides:

- Half of the cross product of the two edges of the triangle.
- Heron's formula (7.2). If the lengths of the sides of the triangle are a, b, and c, then:

$$A = \sqrt{s(s-a)(s-b)(s-c)} \qquad \text{where } s = \frac{a+b+c}{2} \tag{7.2}$$

Once the circles are created, the key aspect of this technique is that, in order to calculate the AO factor for a vertex, no ray casting is needed. Instead, the integrand of Equation (7.1) is approximated by the Equation (7.3), which represent's the accessibility of the vertex E (emitter point) from the vertex R (receiver point):

$$\text{accessibility}(R,E) = 1 - \frac{\text{máx}(0,\cos(\theta_E)) \cdot \text{máx}(0,4\cos(\theta_R))}{\sqrt{\frac{A_R}{\pi} + r^2}} \tag{7.3}$$

The Figure 7.1 represents the terms used in the Equation (7.3). Therefore, the Equation (7.4) is the final formula to calculate the AO factor for a vertex E using the object space AO method, where N is the number of vertices in the model:

$$k_a(E) = \frac{1}{N}\sum_{R=1}^{N} \text{accessibility}(R,E) \tag{7.4}$$

This method is really efficient, as no ray casting is needed. In addition, in order to increase the performance, the accessibility can be stored and only recalculated for the vertices that have changed their position or normal. This way, it also works for dynamic objects.

[1] That way, the area of each triangle is distributed equally among its three vertices.

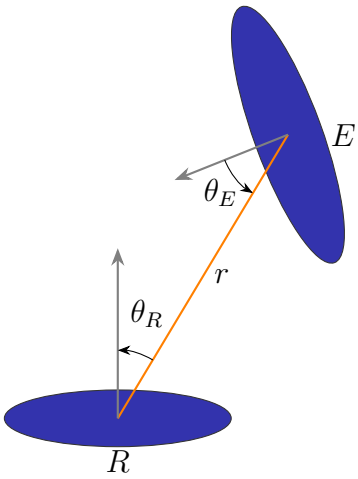

Figura 7.1: Object Space AO: Schematic representation of the accessibility of the vertex E (emitter point) from the vertex R (receiver point).

Shadowed Occluders

This approach faces a problem with the already shadowed occluders, which are the occluders that are themselves in shadow. For example: a point on the surface may have an object nearby that blocks light, but if that object is in turn covered by another geometry, we should not add both shadows. If we did, the point would appear artificially dark. Only the main occlusion should determine the final lighting.

In order to solve this problem, the shadowed occluders should have less influence on the final AO factor. Multiple passes can be used to calculate the AO factor, and in each pass, the AO factor should be updated by multiplying the previous AO factor with the new one calculated in the current pass. This way, the influence of shadowed occluders is reduced, as their contribution to the AO factor will be multiplied by a value less than 1 in subsequent passes.

7.3. Bent Normals

Bent normals are a technique used in ambient occlusion to account for the directionality of light. Instead of just calculating the amount of ambient light that reaches a point on a surface, the normal shading is applied. However, the usual normal is not used, but a bent normal is calculated, which points in the direction of the most unoccluded path from the point on the surface. This allows for more accurate shading, as it takes into account the fact that light may be coming from a specific direction, rather than being uniformly distributed.

This technique obviously faces problems where more than one path is unoccluded, as it only considers the most unoccluded path. Therefore, it does not always provide the most accurate results, especially in complex scenes with multiple light sources and occluders.

7.4. Screen Space AO

Screen Space Ambient Occlusion (SSAO) is a real-time technique used in computer graphics to approximate ambient occlusion effects. It operates in screen space, meaning it firstly renders the scene to a depth buffer and then uses that information to calculate the ambient occlusion for each pixel on the screen[2]. This makes this technique more efficient, as only what is visible on the screen is processed, rather than the entire scene.

For each pixel, SSAO samples the surrounding pixels (creating an sphere around the pixel) and checks the depth values to determine how much of the surrounding geometry is in front of the current pixel, using therefore the `z`-buffer to recreate the profile. The more geometry that is in front, the higher the occlusion factor, which results in a darker pixel. The reader may think that this method introduces a lot of noise, as in the edges between the occluded and non-occluded areas weird patterns may appear. However, this technique produces really good results, and is widely used in real-time applications given that it only processes what is visible on the screen, which makes it much faster than other methods that require processing the entire scene.

7.4.1. Deferred Shading

As mentioned in the previous section, SSAO usually uses deferred shading, which is a rendering technique that separates the rendering process into two main stages: geometry pass and lighting pass.

1. In the geometry pass, the scene is rendered and each of the necessary attributes (such as normals, depth, color, etc.) are stored in different textures, collectively known as the G-buffer. This allows for the storage of all the information needed for lighting calculations without having to re-render the geometry for each light source.

2. In the lighting pass, only the visible pixels are processed. The whole image is covered using just two triangles, and for each pixel, the information stored in the G-buffer is used to calculate the lighting, including the ambient occlusion using SSAO. This way, the lighting calculations are performed only on the pixels that are visible on the screen, which significantly improves performance compared to traditional forward rendering techniques where lighting is calculated for every object in the scene regardless of visibility.

7.4.2. SSAO Extensions

There are various extensions to the basic SSAO technique that aim to improve the quality of the results or to add additional features. Some of them also include using the normals stored in the G-buffer to take into account the directionality of light, which can help to reduce the noise and improve the overall appearance of the ambient occlusion.

[2]This is achieved using Deferred Shading, see Section 7.4.1.

Horizon-Based Ambient Occlusion (HBAO)

Horizon-Based Ambient Occlusion (HBAO) is an extension of the SSAO technique that aims to provide higher quality results. For every pixel, HBAO also uses an integral approach to calculate the ambient occlusion factor, but instead of integrating over the hemisphere, it integrates over the circle that encloses the pixel. For each direction θ in the circle, calculates:

- The tangent angle $T(\theta)$: the angle between the tangent vector and the x axis.
- The horizon angle $H(\theta)$: the angle between the horizon vector and the x axis. The horizon vector is the vector that points to the first occluder in a given direction, which is calculated by sampling the depth buffer along that direction until an occluder is found.

Therefore, the ambient occlusion factor in the pixel p is:

$$AO(p) = 1 - \int_{-\pi}^{\pi} (\sin(H(\theta)) - \sin(T(\theta)))\, W(\theta) d\theta$$

where $W(\theta)$ is just a factor needed so that $AO \in [0, 1]$. Even though it is slower because of the integral, this method is still really efficient because it also uses Deferred Shading. It provides better results than basic SSAO, as it takes into account the geometry of the scene more accurately, but it is also more computationally expensive.

This method also faces problems at the edges of the screen, as the horizon vector may not be properly calculated due to the lack of surrounding pixels. In order to solve this problem, the image is usually rendered in a quite larger scene, and then the edges are cropped to fit the screen. This way, the horizon vector can be calculated properly even at the edges of the screen, which improves the overall quality of the ambient occlusion effect.

8. Terrain Rendering

Terrain rendering is the process of rendering a 3D representation of real terrain (as opposite to terrain synthesis, which is the process of generating a 3D representation of a terrain). It is a crucial aspect of computer graphics, as it is used in several different applications with different requirements:

- Games: High, constant frame rate. The quality is desired to be as good as possible, but it is not the most important aspect.
- Simulators: Constraint frame rate, and a high level of realism and accuracy is required.
- Geographical Information Systems (GIS): The quality and accuracy of the terrain is the most important aspect, and the frame rate is not a concern.

However, all these applications face the same challenge: the data set is usually large to gigantic (terabytes) and increasingly detailed. Working with such large data sets is a challenge, and it is usually not possible to load the entire data set into memory.

8.1. Techniques for Terrain Rendering

Usuall, the terrain is represented using two different textures:

- Height map: A 2D texture that stores the height of the terrain at each point. It is usually a grayscale image, where the intensity of each pixel represents the height of the terrain at that point.
- Color map: A 2D texture that stores the color of the terrain at each point. It is usually a RGB image, where the color of each pixel represents the color of the terrain at that point.

However, more information can be stored, such as the normal map, cloud coverage map, etc. Having all this information would allow us to render the terrain as we have seen in previous chapters, but it would not be efficient. Therefore, we need to use different techniques to render the terrain efficiently.

8.1.1. Terrain Ray Tracing

Terrain ray tracing is a technique that allows us to render the terrain efficiently by tracing rays from the camera to the terrain. However, detecting the intersection

of the rays with the terrain is really computationally expensive, and it is not possible to do it in real time. Therefore, a hierarchical data structure is used to accelerate the ray tracing process.

- A *quadtree* is a hierarchical data structure that divides the 2D space into four quadrants. Each quadrant can (but is not required to) be further subdivided into four quadrants, and so on.
- A *octree* is the generalization of a quadtree to 3D space. It divides the 3D space into eight octants, and each octant can (but is not required to) be further subdivided into eight octants, and so on.
- A *kD tree* is another hierarchical data structure that divides the space into two halves at each level. It is a binary tree, where each node represents a splitting plane that divides the space into two halves.

Therefore, the ray tracing process can be accelerated by using an *octree* or a *kD tree* to store the terrain data, where large bounding boxes are used to represent the space where there is no terrain, and smaller bounding boxes are used to represent the space where there is terrain, so that it encloses the terrain as tightly as possible. This way, and given that the intersaction of a ray with a bounding box is much cheaper than the intersection of a ray with the terrain, we can quickly discard large portions of the terrain that are not visible, and only check for intersections with the terrain in the areas that are visible.

8.1.2. Rendering Polygonal Models

Another technique to render the terrain is to use a polygonal model, where the terrain is represented as a mesh of triangles.

- The first implementation that the reader might think of is to use triangles (the `GL_TRIANGLES` primitive) to represent the terrain, where each triangle represents a small portion of the terrain. However, this approach is not efficient with regards to memory. A vertex is not usually used by only one triangle, but by several triangles (usually 6), and therefore, we would need to store the same vertex multiple times, which is a waste of memory.
- A better approach is to use triangle strips (the `GL_TRIANGLE_STRIP` primitive), where each triangle shares two vertices with the previous triangle. This way, we can reduce the amount of memory needed to store the terrain, as we only need to store each vertex once, and we can reuse it for multiple triangles.

This technique has different optimizations that can be applied to improve the performance of the rendering process, as the following ones.

Mesh decimation

Mesh decimation is a technique that allows us to reduce the number of triangles in the mesh, while preserving the overall shape of the terrain. This can be applied in flat regions, where the terrain does not change much, and therefore, we can use

fewer larger triangles to represent the terrain. This technique is not suitable for areas with a lot of detail, as it would result in a loss of detail, but it can be used in flat regions to reduce the number of triangles and improve the performance of the rendering process.

Level of detail (LOD)

Level of detail (LOD) is a technique that allows us to use different levels of detail for different parts of the terrain, depending on the distance from the camera. For example, we can use a high level of detail (with more triangles) for the parts of the terrain that are close to the camera, and a low level of detail (with fewer triangles) for the parts of the terrain that are far from the camera, as they will not be visible in much detail. This way, we can reduce the number of triangles that need to be rendered, and therefore, improve the performance of the rendering process.

Real-time Optimally Adapting Mesh (ROAM)

Real-time Optimally Adapting Mesh (ROAM) is a technique that allows us to adapt the mesh in real time, depending on the distance from the camera and the level of detail required. The mesh is continously split until no contribution is added. This technique implements both the mesh decimation and the level of detail techniques:

- Mesh decimation:

  In order to decide if a division contributes, the difference between the height of the new vertex and the height of the vertexes that surround it is calculated, and if the difference is greater than a certain threshold, then the division is performed, otherwise, it is not performed.

- Level of detail (LOD):

  That treshold can be calculated as a function of the distance from the camera, so that the closer the vertex is to the camera, the smaller the threshold is, and therefore, the more likely it is that the division will be performed; and the farther the vertex is from the camera, the larger the threshold is, and therefore, the less likely it is that the division will be performed.

As we can see, this technique allows us to adapt the mesh in real time, and therefore, it can be used to render the terrain efficiently, as it reduces the number of triangles that need to be rendered, while preserving the overall shape of the terrain. This technique uses a *Triangle Binary Tree* to store the mesh, where each node represents a triangle, and the children of a node represent the triangles that are created by splitting the parent triangle. This way, we can easily perform the mesh decimation and the level of detail techniques, as we can easily access the triangles that need to be split or merged.

A problem that can arise when splitting the triangles is the T-junction problem, which occurs when a triangle is split, and the new vertex is not shared by any other triangle, which can result in visual artifacts. This is a problem, because:

- Given floating point precision, the new vertex might not be exactly on the edge of the triangle, and therefore, holes can appear in the mesh, which can result in visual artifacts.
- Interpolation problems can arise, as the new vertex might not be shared by any other triangle, and therefore, the interpolated color next to the new vertex might not be correct, which can result in visual artifacts.

In order to solve this problem, two different techniques can be used:

- Recursive splits: When a triangle is split, as many successive splits as necessary are performed, until no T-junctions are created.
- Geomorphing: When a triangle is split, the new vertex is slightly moved to the inside of the triangle, so that instead of creating a T-junction, a small triangle is created, which can be rendered without visual artifacts.

Even though the ROAM technique produces good results, dynamic real-time mesh adaptation is too expensive for modern graphics hardware, and therefore, it is not so used in practice.

8.2. GPU-Friendly Terrain Rendering

The techniques that we have seen so far are not very GPU-friendly, as they require a lot of CPU-GPU communication, and they are not suitable for modern graphics hardware. Therefore, we need to use different techniques that are more suitable for modern graphics hardware, such as the following ones.

8.2.1. Geometry Clipmaps

Geometry clipmaps is a technique that allows us to render the terrain efficiently by using a hierarchical data structure that is stored in the GPU. The terrain is represented as a set of *nested grids*, similar to a pyramid mipmap, where each grid represents a different LoD of the terrain. Each level covers much more area than the previous one, but with less detail. The grid is centered around the camera.

When the camera moves, only some information needs to be updated, as the grid is centered around the camera, and therefore, only the information that is outside the grid needs to be updated. This way, not the entire terrain needs to be updated, but only a small portion of it, which can be done efficiently in the GPU.

Some of its advantages are that, given that the structure of the nested grids is regular, it enables fast rendering using triangle strips. In addition, only regular and continuous chunks of memory have to be read, which is also efficient. However, only LoD and no mesh decimation depending the hight of the terrain is implemented, so more triangles than necessary are rendered.

8.2.2. Tile-based, restricted Quadtree

Tile-based, restricted quadtree is a technique that allows us to render the terrain efficiently by using a quadtree data structure that is stored in the GPU. The original

terrain (which is a texture) is split into tiles. The root node of the quadtree represents the entire terrain, and each child node represents a quadrant of the parent node, but with a higher level of detail.

This way, a regular mesh is created. To store this mesh on the GPU efficiently, vertex compression via linearization of the restricted quadtree can be used. This technique stores the mesh in a compact form, reducing the amount of memory required. The mesh is linearized by depth-first traversing the quadtree and generating a continuous triangle strip, where each new triangle only requires adding one additional vertex.

8.2.3. S3TC (DXT*)

S3 Texture Compression (S3TC) is a technique that allows us to compress textures. It is a lossy compression technique (it reduces the quality of the texture), but it decoding is very fast, and it can be done in the GPU. The texture is divided into blocks of 4x4 texels, and each block is compressed independently.

- Originally, in a texture that uses 8 bits per channel, each texel would require $3 \cdot 8 = 24$ bits.
- With S3TC, each block of 4x4 texels (16 texel) is compressed into 64 bits, which means that each texel requires only $64/16 = 4$ bits, which is a compression ratio of 6:1.

For each block, only four colors are considered, two of them are representative colors, and the other two are obtained by interpolating the two representative colors.

- $c_0(00)$: One of the representative colors.
- $c_1(01)$: The other representative color.
- $c_2(10)$: An interpolated color, which is closer to c_0.
- $c_3(11)$: Another interpolated color, which is closer to c_1.

For each of the 16 texels, one of the four colors is chosen, and therefore, we need 2 bits to store the color of each texel. This means that we need $16 \cdot 2 = 32$ bits to store the color of the 16 texels. In addition, each of the representative colours is represented using 16 bits[1], which means that the they require the other 32 bits.

The remaining challenge is how to choose the representative colors, and how to decide which of the four colors to use for each texel. Two different techniques can then be used to compress the texture.

[1]The RGB 565 format is used, where 5 bits are used for the red channel, 6 bits for the green channel, and 5 bits for the blue channel. This format is used because the human eye is more sensitive to green than to red and blue.

Range fit

All the colors in the block are represented in a 3D color space (where each axis represents one of the color channels).

1. First of all, a main axis is calculated, which is the axis that best fits the distribution of the colors in the block.

2. Then, the colors are projected onto the main axis, so now we have a 1D distribution of the colors.

3. The representative colors are chosen as the two colors whose projections are the farthest apart, so that they can represent the colors in the block as well as possible. They are then two of the colors that the block contains.

4. A line is drawn between the two representative colors, and the other two colors are obtained by interpolating the two representative colors, so that they are equally spaced on the line. This way, the four colors are equally spaced on the line, and therefore, they can represent the colors in the block as well as possible. It should be noted that these two new colors might not be colors that the block contains, but they are still good representative colors for the block.

5. Finally, for each of the 16 texels, the color that is closest to the color of the texel is chosen.

This technique is simple and fast, but it does not always produce good results. If a color is an outlier, it can affect the main axis and the representative colors, which can result in a poor representation of the colors in the block.

Cluster fit

Cluster fit is a technique that allows us to find the representative colors by clustering the colors in the block. As happens with the range fit technique, the colors are represented in a 3D color space.

1. First of all, a main axis is calculated, which is the axis that best fits the distribution of the colors in the block.

2. Each of the colors in the block is projected onto the main axis, so now we have a 1D distribution of the colors.

3. The part of the main axis where there are projections is divided into four equal parts, which represents four different clusters. For each of the 16 colors, one of the four clusters is chosen, depending on which of the four parts of the main axis the projection of the color falls into.

4. The representative color of each cluster is calculated as the average of the colors in the cluster, so that it can represent the colors in the cluster as well as possible. It should be noted that the representative colors might not be colors that the block contains, but they are still good representative colors for the block.

This technique is more complex and slower than the range fit technique, but it can produce better results, as it is less affected by outliers, as the representative colors are calculated as the average of the colors in the cluster.

8.3. Rendering Execution Methods

There are two different rendering execution methods that can be used to render the terrain: *rastering* and *ray casting*.

8.3.1. Rastering

Rastering is the traditional rendering method, where the terrain is represented as a mesh of triangles, and the triangles are rasterized to produce the final image. This method is efficient for rendering polygonal models. The time complexity of this method, where T is the number of triangles and F is the number of fragments, is:

$$t_{\text{Rasterizer}}(T, F) = O(T) + O(F)$$

However, usually the fragment processing is not considered, as no interpolation is needed (everything is stored in textures), and therefore, the time complexity is usually considered to be $O(T)$:

$$t_{\text{Rasterizer}}(T) \approx O(T)$$

Therefore, this method is efficient when the number of triangles is not too large.

8.3.2. Ray Casting

Ray casting is a rendering method where rays are cast from the camera to the terrain, and the color of each pixel is determined by the intersection of the ray with the terrain. This method is efficient when a Quadtree or an Octree is used to store the terrain data, as it allows us to quickly discard large portions of the space where there is no terrain, and therefore, no intersection can occur. The time complexity of this method, where P is the number of pixels and S is the number of steps taken to find the intersection of a ray with the terrain, is:

$$t_{\text{RayCaster}}(P, S) = O(P \cdot S)$$

However, usually the number of steps is considered to be constant, so the time complexity is usually considered to be $O(P)$:

$$t_{\text{RayCaster}}(P) \approx O(P)$$

Therefore, this method is efficient when the number of pixels is not too large.

8.3.3. Hybrid Method

The hybrid method is a rendering method that combines the rastering and ray casting methods, using the one that is more efficient for each part of the terrain. For example, representing a forest might involve a small part of the scene (low number of pixels) but a big number of triangles, so ray casting would be more efficient; while representing a flat terrain might involve a big part of the scene (high number of pixels) but a small number of triangles, so rastering would be more efficient. This way, we can take advantage of the strengths of both methods, and therefore, we can render the terrain efficiently.

9. Terrain Synthesis

Terrain synthesis is the process of generating realistic terrain models for various applications, such as video games, simulations, and virtual reality. The goal is to create visually appealing and believable landscapes that can be used in a variety of contexts. The proterties that the generated terrain should have include:

- Non-repetitive: The terrain should not have obvious repeating patterns.
- Infinite extent: The terrain should be able to extend indefinitely in all directions.
- Interactive frame rates: The terrain should be generated and rendered efficiently to allow for real-time interaction.
- Real-time editing: The terrain should be editable in real-time, allowing users to modify the landscape as needed. Since the terrain is generated, the programmer should be able to modify the terrain generation parameters in real-time, allowing for dynamic changes to the landscape.

All of these properties can be achieved through *fractal terrain synthesis*. This is a method of generating terrain using fractal algorithms. Fractals are mathematical objects that exhibit self-similarity at different scales, making them ideal for modeling natural phenomena such as mountains, valleys, and coastlines. The fractal terrain synthesis process is known as *Rescale-and-add*.

9.1. Rescale-and-add

The rescale-and-add process is a method of generating terrain by repeatedly rescaling and adding a base image. This process creates a fractal-like structure that can produce realistic terrain features. The process is iterative and involves the following steps:

1. Start with a simple base, such as simple noise of some filter.
2. Rescale the base to half the size of the original.
3. Rescale the rescaled base to half its size.
4. ...
5. The smaller the size, the more detail is added to the terrain. This process is repeated until the desired level of detail is achieved.

6. The final terrain is obtained by summing all the rescaled versions, but the smaller will have more influence on the final terrain than the larger ones. This is because the smaller versions add more detail to the terrain, while the larger versions provide a rough outline of the landscape.

The rescale-and-add process can be implemented using the Equation (9.1), which represents the height of the terrain at a given point (x, y) as a sum of rescaled versions of the base image:

$$H(x,y) = \sum_{i=0}^{n} \frac{1}{2^{n-i}} \cdot F\left(\frac{1}{2^i}x, \frac{1}{2^{n-i}} \cdot y\right) \tag{9.1}$$

The function F represents the base image, which can be generated using various different functions, such as filtered white noise, Perlin noise, images... The choice of the base image will affect the final appearance of the terrain, as it determines the overall structure and features of the landscape.

9.1.1. Parameters and their effects

In fact, the rescale-and-add process has four parameters that can be adjusted to achieve different types of terrain:

- Base function F.
- Roughness r: This parameter controls the influence of the smaller rescaled versions on the final terrain.
- Octaves o: This parameter determines the number of rescaled versions that are added together to create the final terrain.
- Lacunarity l: This parameter controls the rescaling factor between successive octaves.

Using those parameters, we can create a wide variety of terrains, from smooth rolling hills to rugged mountains and deep valleys. The Equation (9.2) represents the height of the terrain at a given point (x, y) with the parameters included:

$$H(x,y) = \sum_{i=0}^{o} \frac{1}{2} \cdot r^{i-1} \cdot F\left(\frac{1}{l^{o-i}}x, \frac{1}{l^{o-i}}y\right) \tag{9.2}$$

We can observe that the Equation (9.1) is a special case of the Equation (9.2) when $r = 0{,}5$, $o = n$ and $l = 2$.

9.1.2. Multifractals

Multifractals are a generalization of fractals that can model more complex and heterogeneous structures. In the context of terrain synthesis, multifractals can be used to create terrains with varying degrees of roughness and detail across different regions. In order to achieve this, we can have different base functions $F_i(x, y)$, each one with its own weight $a_i(x, y)$, which determines the influence of each base function on each point of the terrain. The functions a_i have to satisfy the following conditions forall point (x, y):

1. $a_i(x, y) \geqslant 0 \qquad \forall i$, which means that the weights must be non-negative.
2. $\sum_i a_i(x, y) = 1$, which means that the weights must sum to 1 at each point.

Under these conditions, the function F of the Equation (9.2) can be defined as a weighted sum of the base functions:

$$F(x, y) = \sum_i a_i(x, y) \cdot F_i(x, y) \tag{9.3}$$

This allows for the creation of different types of terrain in different regions, as the weights can be adjusted to give more influence to certain base functions in specific areas.

10. Fur

Plenty of different objects in our real world are made of fur: animals, carpets, coats... even people's hair is considered fur. Rendering fur is a challenging problem in computer graphics, as it requires simulating the complex interactions of light with the individual strands of fur. In this chapter, we will explore various techniques for rendering fur.

10.1. Geometric Representation of Fur

One common approach to rendering fur is to represent it using geometric primitives. The most common primitives used for this purpose are lines and cylinders or cones.

- Cones/Cylinders: Each strand of fur can be represented as a cone or cylinder, with the base at the root of the fur and the tip at the end. This allows for a more realistic representation of the fur's thickness and tapering. However, this approach is computationally expensive on current hardware. Taking into account the large number of strands required to create a realistic fur effect, rendering each strand as a cone or cylinder can be very resource-intensive.
- Lines: It creates a more interactive representation, used specially with sparse fur. However, it has poor filtering and the collision tests are expensive.

10.2. Texture-Based Fur Rendering

Another approach to rendering fur is to use texture-based techniques. Two main approaches are the *volume textures* and the *shell textures*.

10.2.1. Volume Textures

Volume textures are 3D textures that store information about the fur's density and color. They can be used to create a representation of fur. However, the result is not very realistic, as it does not capture the individual strands of fur and their interactions with light; it is just a sticked-on texture.

10.2.2. Shell Textures

Shell textures are a more advanced technique that involves layering multiple textures to create a more realistic representation of fur. A base shell texture is

created to represent the overall shape and density of the fur, and then multiple layers are added on top (regularly spaced) to add detail and depth. This technique can produce more realistic results than volume textures, as it allows for the simulation of individual strands of fur and their interactions with light. The process of generating the shell texture has the following steps:

1. Seed the surface with curl starting points.

2. Simulate the curl growth by using a physics-based model to determine how the strands of fur will grow and curl over time.

3. Interpolation is used to create more strands of fur between the seed points, resulting in a denser and more realistic fur effect. Hair-to-hair collisions are ignored.

4. The volume is sampled to obtain the color, opacity and the normal of the fur at each point, which are then used to create the final shell texture.

However, three main issues arise with this technique:

- Surface Parameterization: Given an arbitrary surface, it is not always easy to find a good parameterization for the fur. In 2D, it is easy to find a parameterization for a surface, but in 3D, it can be more difficult to find a parameterization that works well for the fur.

- Texture memory usage: Several shells are used, over the entire surface, and at hair resolution, which can lead to high memory usage.

- Poor Silhouettes: The silhouette of the fur can look unrealistic, as the shells break apart at oblique angles, creating a jagged and unnatural appearance instead of a smooth silhouette.

Lapped volume textures

This solution solves two problems: the surface parameterization and the memory usage. It should be noted that this solution is notonly for shell texture, but it can be applied to any volume texture. This approach has the following steps:

1. Firstly, a texture patch is created. A texture patch is a small representative section of the volume texture.

2. Then, the local orientation of the surface is computed. This involves determining the direction of the surface (usually using the normal and tangent vector), in order to know how to orient the texture patch on the surface.

3. Once the local orientation is computed, the texture patch is applied to the surface, repeating it as necessary to cover the entire surface. The texture patch is oriented according to the local orientation of the surface, ensuring that it aligns correctly with the surface's features.

Everything is stored in a *texture atlas*, which is a large 2D texture that contains all the surface. This way, two different points on the surface will not share the same texture coordinates, which allows for a more efficient use of memory and better performance. This way, the parameterization problem is solved and the memory usage is reduced, as only a small representative section of the volume texture is stored in the texture atlas, rather than the entire volume texture.

Fin textures

This solution solves the silhouette problem. It is a technique that involves using a series of fin-like structures to create a more realistic silhouette for the fur. The fins are created by extruding the textures but in a perpendicular direction to the surface, rather than parallel to it. Depending on the viewing angle, the fins can be faded out to create a more realistic appearance. This allows for a more natural and smooth silhouette.

Rendering

In order to render the fur using this technique and the solutions mentioned above, the rendering order is as follows:

1. Render the skin. This is the base layer of the fur, and it provides the underlying color and texture for the fur.

2. Render the fin textures.

3. Render the shell textures.

Both the fin textures and the shell textures should be rendered with alpha blending (in order to create a more realistic appearance), and with the depth write disabled.

10.3. Hair Simulation

In order to simulate human hair, a constrained particle-based hair simulation is used. These are the needed steps:

1. Some guided hair strands are created, which are used to define the overall shape of the hair. They are defined as bezier curves, which allows for a smooth and natural appearance.

2. The guide strands are then tessellated into a set of stright lines (which is the primitive used for rendering the hair). Tessellation is the process of breaking down the guide strands into smaller segments, which can be rendered.

3. The hair strands are then interpolated between the guide strands, creating a dense and realistic hair effect. This involves using the guide strands as a reference to create new strands of hair that are similar in shape and direction to the guide strands.

10.3.1. Grid-based fluid simulation

In order to simulate correct hair dynamics, a grid-based fluid simulation is used. Two ping-pong volumes are used, which means that two volumes are used to store the hair simulation data, and they are alternately read from and written to during the simulation process. This allows for efficient memory usage and better performance.

First of all, a (precomputed) mesh collision volume (or analytic body description) is created, which is used to detect collisions between the hair and the body in an easy and efficient way. After that, the guide hairs are voxelized (converted into a grid-based representation). For each voxel, the following forces are computed:

- Wind forces
- Gravity forces
- If the voxel is colliding with the body, then a collision response force is computed so that the hair does not penetrate the body.

However, and given that no hair-to-hair collisions are simulated, the hair strands can pass through each other and may appear too dense or tangled. To address this issue, a last artificial force is added to the simulation in the *negative direction of the hair density gradient*. This force helps to spread the hair strands apart and create a more natural and realistic appearance, perserving the hair volume.

10.4. Example: Ratatouille

Ratatouille is an example of a movie that used the techniques described in this chapter to create realistic fur for the characters. They used cylinders to represent the fur with the following techniques:

- Perfect Bounding Boxes: They used perfect bounding boxes to optimize the rendering of the fur, which allowed them to efficiently detect collisions and occlusions.
- Level of Detail (LOD): They used LOD techniques to reduce the complexity of the fur rendering depending on:
  - Distance: The further the character is from the camera, the less detail is needed for the fur.
  - Motion blur: When the character is moving quickly, less detail is needed for the fur, as it will be blurred in the final image.
  - Depth of field: When the character is out of focus, less detail is needed for the fur, as it will be blurred in the final image.

11. HDRI

High Dynamic Range Imaging (HDRI) is a technique used in photography and computer graphics to capture and represent a wider range of luminosity than what is possible with standard imaging techniques. HDRI allows for the representation of both very bright and very dark areas in a scene, which can be particularly useful in situations where there is a significant contrast between light and shadow. In real live, dark colors can be 10^4 times darker than bright colors ($1 : 10000$), while normal images that use 8 bits per channel can only represent a range of $1 : 256$. During this chapter, we will first discuss how does the human work, and then the three main stages of HDRI will be covered: recording, storage, and output.

11.1. Nature

In order to understand HDRI, it is important to first understand how the human eye perceives light and color. Light first enters the eye through the cornea, a first lens that protects the eye and helps to focus light. The light then passes through the pupil, which is a hole that can adjust its size to control the amount of light that enters the eye. The iris, which is the colored part of the eye, controls the size of the pupil. After the light passes through the pupil, it is focused by the lens onto the retina, which is a layer of light-sensitive cells (photoreceptors) at the back of the eye. The information captured by the photoreceptors is then transmitted to the brain via the optic nerve, where it is processed and interpreted as visual information.

11.1.1. Photoreceptors

There are two main types of photoreceptors in the human eye: rods and cones.

Rods are responsible for vision in low-light (scotopic vision) conditions. They are highly sensitive to light but do not provide color information. Rods are more numerous ($120 \cdot 10^6$) than cones and are primarily located in the peripheral regions of the retina.

Cones are responsible for vision in bright-light (photopic vision) conditions and for color perception. They are less sensitive to light than rods but can detect a wide range of colors. Cones are concentrated in the central part of the retina, particularly in the fovea, which is responsible for sharp central vision. There are much less cones ($6 \cdot 10^6$) than rods in the human eye. There are three types of cones, each sensitive to different wavelengths of light:

- S-cones (short-wavelength cones). Only 2 % of the cones are S-cones, which are most sensitive to blue light.
- M-cones (medium-wavelength cones). About 32 % of the cones are M-cones, which are most sensitive to green light.
- L-cones (long-wavelength cones). The majority of cones, about 64 %, are L-cones, which are most sensitive to red light.

As already mentioned, the retina has a zone called *fovea* where cones are concentrated. The fovea is responsible for sharp central vision and is crucial for tasks that require detailed visual information, such as reading and recognizing faces. However, no rods are present in the fovea, which means that our central vision is not sensitive to low-light conditions.

11.1.2. Chromatic Aberration

Chromatic aberration is a common optical problem that occurs because different wavelengths of light are refracted (bent) by different amounts as they pass through a lens. This can lead to a situation where different colors of light do not converge at the same point after passing through the lens, resulting in a blurred or distorted image. The eye's lens is designed to minimize chromatic aberration, but it can still occur, particularly in peripheral vision where the lens is less effective at correcting for it. However, in high-quality photography cameras, chromatic aberration is avoided by using two or more lens elements made of different types of glass, which can help to correct for the different refraction of light and reduce the effects of chromatic aberration.

11.1.3. Dynamic Range

A high quality camera that stores 14 bits per channel can represent a range of $1 : 16384$. However, the human eye can perceive a much wider dynamic range, estimated to be around $1 : 10^6$ under ideal conditions. This means that the human eye can see details in both very bright and very dark areas of a scene simultaneously. There are three different light types:

- Photopic vision: This is the vision we use in bright light conditions, where cones are active.
- Scotopic vision: This is the vision we use in low light conditions, where rods are active.
- Mesopic vision: This is the vision we use in intermediate light conditions, where both rods and cones are active.

11.2. Recording

Back to our main topic, the first stage of HDRI is recording, which involves capturing the wide range of luminosity in a scene. HDR cameras can be used, but they are really expensive. A more common approach is to use a technique called

exposure bracketing, which involves taking multiple photographs of the same scene at different exposure levels. This is however not suitable for dynamic scenes as the objects in the scene may move between shots, leading to misalignment and ghosting artifacts.

11.3. Storage

There are two main color models used when working with colors.

- Additive Color: This model is based on the principle of adding light to create colors. It is used in devices that emit light, such as computer screens and televisions. In this model, the primary colors are red, green, and blue (RGB).
- Subtractive Color: This model is based on the principle of subtracting light to create colors. It is used in devices that reflect light, such as printers and paints. In this model, the primary colors are cyan, magenta, and yellow (CMY).

Therefore, we will focus on the RGB color model, which is the one used in computer graphics and digital imaging. There are different *spaces* which are used to represent colors in the RGB model, such as:

- RGB: This is the most common color space used in digital imaging and computer graphics. It represents colors as a combination of red, green, and blue values.
- CMYK: This color space is used copying the way it would be generated by a printer. It represents colors as a combination of cyan, magenta, yellow, and black values.
- XYZ: This color space is based on the CIE 1931 color space, which is a mathematical model that describes how the human eye perceives color. It represents colors as a combination of three values: X, Y, and Z, which correspond to the three types of cones in the human eye.
- HSV: This color space represents colors in terms of hue, saturation, and value (brightness). It is often used in applications where it is more intuitive to manipulate colors based on their hue and saturation rather than their RGB values.

Each of these spaces can represent the same colors, but they use different ways. There are transformation formulas (usually matrixes) that can be used to convert colors from one space to another. In addition, there are different color gamuts, which are the range of colors that an specific technique can represent. Some of them are represented in the Figure 11.1.

Regarding for the storage of HDR images, usually no fixed black and white points are used (as opposed to LDR images). In order to achieve better precision regarding luminance, two approaches are commonly used:

- Save luminance for the 3 channels in high precision.

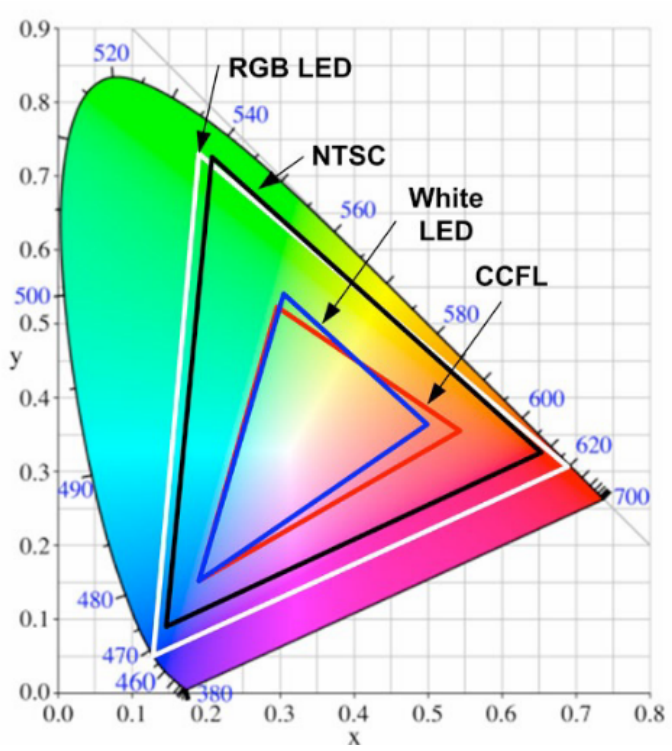

Figura 11.1: Different color gamuts.

- Save the luminance in a separate channel in high precision, and the color information in the other 3 channels in low precision.

In order to save the HDR images, there are different file encoding schemes, such as:

- Pixar Log Encoding (`.tiff`): it uses 11 bits per channel, and it is a logarithmic encoding that allows for a wide dynamic range while maintaining good precision in the mid-tones.
- it uses 8 bits for each of the RGB channels to represent the mantissa, and 8 bits for a shared exponent that allows for a wide dynamic range.
- SGI LogLuv (`.tiff`): it uses 16 bits for logarithmic luminance, and the color information is stored in two 8 bit channels for the u and v chromaticity coordinates.
- ILM OpenEXR (`.exr`): it uses 16 bit floating point format for each channel, which allows for a wide dynamic range and high precision in representing colors and luminance.

11.4. Output

In order to display HDR images there are also two main approaches. The first one is to use an specific HDR display, which is capable of displaying a wider range of luminosity than standard displays but is however really expensive; while the second

one is to use a technique called *tone mapping*. Tone mapping is a process that takes the HDR image and maps it to a LDR image that can be displayed on a standard monitor. The main goal that lies behind tone mapping is to reproduce visibility: if an object is seen in the HDR image, it should also be seen in the LDR image. Some naïve tone mapping techniques include:

- Setting to the maximum value of the LDR image the brightest pixel in the HDR image, and then scaling all the other pixels linearly. This would produce an almost black image, as the light sources are multiple orders of magnitude brighter than the rest of the scene.
- All the light sources in the HDR image are set to the maximum value of the LDR image. After that, the brightest non light source pixel in the HDR image is set to the maximum value of the LDR image minus ε (where ε is a small value) and all the other pixels are scaled linearly. In this case, visibility is preserved but experience wouldn't, as changes in the light emission would not be visible (day and night would look the same).

The technique that should then be applied is called *histogram equalization*, which is a method that adjusts the histogram of the image.

Definición 11.1 (Histogram)**.** The histogram of an image is a graphical representation of the distribution of pixel intensity values in the image. It shows the number of pixels for each intensity level, which in 8 bit images ranges from 0 (black) to 255 (white).

An ideal histogram for an image would be a histogram where there are pixels in all the intensity levels, which would mean that the image has a good contrast and a wide range of tones. If the extremes of the histogram are empty, it means that the image range is not fully utilized, which should be avoided by applying histogram equalization. This technique works by redistributing the pixel intensity values in the image so that they are more evenly distributed across the available range. This can help to improve the contrast and visibility of details in the image, particularly in areas that are too dark or too bright. This idea is achieved using the *Cumulative Distribution Function* (CDF) of the histogram, which is a function that gives the cumulative number of pixels for each intensity level, i.e., the number of pixels with intensity less than or equal to a given intensity level. The CDF of a good histogram should be a straight line that goes from 0 to the total number of pixels, which means that the pixel intensity values are evenly distributed across the available range. The histogram equalization technique works using the Equation 11.1, where:

- v is the original pixel intensity value.
- $h(v)$ is the new pixel intensity value after histogram equalization.
- $CDF(v)$ is the cumulative distribution function of the histogram at intensity level v.
- CDF_{min} is the minimum value of the cumulative distribution function, which corresponds to the lowest intensity level in the image.

- (height · width) is the total number of pixels in the image.

- L is the number of possible intensity levels in the image (for 8 bit images, $L = 256$). $L - 1$ is used because the intensity levels are indexed from 0 to $L - 1$.

$$h(v) = \frac{CDF(v) - CDF_{min}}{(\text{height} \cdot \text{width}) - CDF_{min}} \cdot (L - 1) \tag{11.1}$$

It should be noted that the first part of the Equation 11.1 is used to normalize the CDF values to the range $[0, 1]$, while the second part scales the normalized values to the range of intensity levels in the image. By applying this transformation to each pixel in the image, we achieve a more balanced distribution of pixel intensity values, which enhances the overall contrast and visibility of details in the image.

11.4.1. Light Bloom

When light passes through a circular aperture (as in the case of a camera lens), it creates a diffraction pattern known as an Airy disk. This pattern consists of a central bright spot surrounded by concentric rings of decreasing intensity. This phenomenon can cause a visual effect called "light bloom" or "veiling", where bright light sources appear to bloom or spread out beyond their actual boundaries. This is barely noticeable in smooth lighting conditions, but may become more pronounced in scenes with high contrast where bright regions are adjacent to dark regions.

Synthetic images do not suffer from light bloom, as they are generated using mathematical models that do not account for the physical properties of light and optics. However, in order to achieve a more realistic rendering, some rendering techniques may simulate light bloom by applying a post-processing effect that mimics the diffraction pattern created by a circular aperture. The steps to achieve this effect are as follows:

1. Apply a bright-pass filter to the luminance image, which isolates the bright areas of the image that will contribute to the bloom effect.

2. Blur the output of the bright-pass filter using a Gaussian low-pass filter, which simulates the spreading of light caused by diffraction.

3. Additively blend the blurred output onto the original tone-mapped image, which creates the final image with the bloom effect applied. This blending step allows the bright areas to appear as if they are blooming or spreading out, while still preserving the details in the rest of the image.

The light bloom effect can enhance the visual appeal of an image by adding a sense of realism and depth, particularly in scenes with bright light sources such as sunlight, streetlights, or glowing objects. However, it should be used judiciously, as excessive bloom can lead to a loss of detail and make the image appear washed out or overly bright.

11.4.2. Star Effect

The star effect, also known as the starburst effect, is a visual phenomenon that occurs when bright light sources are captured in an image. It creates a star-like pattern of rays emanating from the light source, which can add a dramatic and artistic touch to photographs. The star effect is typically caused by diffraction of light around the edges of the camera's aperture blades.

In synthetic images, the star effect can be simulated by applying a post-processing effect that mimics the diffraction pattern created by the aperture blades. The steps to achieve this effect are as follows:

1. Apply a bright-pass filter to the luminance image, which isolates the bright areas of the image that will contribute to the star effect.

2. Blur the output of the bright-pass filter using a directional blur that simulates the diffraction pattern created by the aperture blades. The direction and intensity of the blur can be adjusted to create different star patterns. It may be necessary to apply multiple directional blurs at different angles to create a more complex star pattern, especially if the aperture has multiple blades.

3. Additively blend the blurred output onto the original tone-mapped image, which creates the final image with the star effect applied. This blending step allows the bright areas to appear as if they are emitting rays of light, while still preserving the details in the rest of the image.

11.4.3. Real-time Tone Mapping

In real-time applications, such as video games or interactive simulations, it is crucial to perform tone mapping efficiently to maintain a high frame rate. The steps for real-time tone mapping are as follows:

1. Render the illuminated scene into floating point texture, which allows for a wide dynamic range and high precision in representing colors and luminance.

2. Rescale the rendered image to a smaller size to create a downsampled version of the image, which helps improving the performance of the subsequent steps.

3. Determine the average luminance of the downsampled image using logarithmic reduction.

4. Apply the bright-pass filter and then post-process the image to apply the desired effects, such as light bloom or star effect, as described in the previous sections.

5. Apply the tone mapping operator to the original rendered image.

6. Additively blend the post-processed bloom image onto the tone-mapped image to create the final output that will be displayed on the screen.

This approach allows for efficient tone mapping while still achieving high-quality results, making it suitable for real-time applications where performance is a concern. By downsampling the image and using logarithmic reduction to calculate the average luminance, the computational load is reduced, allowing for faster processing and smoother performance in interactive environments.

12. REYES

"Render Everything You Ever Saw" (REYES) is a rendering algorithm developed by Lucasfilm Graphics group in the 80s, used by PIXAR, and described in the paper "The Reyes Image Rendering Architecture". It was used in many of Pixar's films, and in order to present it, they used the well known "The Road to Point Reyes" image[1], which is shown in the Figure 12.1. Even though it may appear to be unrealistic, it was really impresive at the time.

REYES is an architecture optimized for fast high-quality rendering of complex animated scenes.

- Fast: being able to compute a feature-length film in approximately a year. This is still valid today, as the computational power has increased significantly but the complexity of the scenes has also increased.
- High-quality: virtually indistinguishable from live action motion picture photography.
- Complex: visually rich as real scenes.

The REYES architecture has some assumptions and goals that are worth mentioning:

- Model complexity: given that the scenes are complex, they expected the scenes to have hundreds of thousands of geometric primitives.
- Model diversity: it was needed support for a large variety of geometric primitives, especially data amplification primitives such us procedural models (for instance, as happens with hair, obtaining everything just from a couple of guide hairs), fractals and particle systems.
- Shading complexity: a programmable shader was needed.
- Minimal Ray Tracing: many non-local lighting effects can be approximated with texture maps. Few objects in natural scenes would seem to require ray tracing.
- Speed: assuming 24 fps, in order to render a 2 h film in a year, the rendering time per frame should be around 3 min.
- Image Quality: aliasing and faceting artifacts should be avoided.
- Flexibility: given that new techniques were going to be discovered, this architecture should be flexible enough to accomodate them.

[1] Point Reyes is actually a national park in California, where this algorithm was developed.

Figura 12.1: The Road to Point Reyes, rendered using the REYES algorithm.

12.1. Design Principles

The assumptions previously described led us to a set of architectural design principles.

- Natural Coordinates: Each calculation should be done in a coordinate system that is natural for that calculation. For instance, texture mapping should be done in the coordinate system of the local surface geometry, while the visible surface calculations are most naturally done in pixel coordinates (screen space).

- Vectorization: The architecture should be able to exploit vectorization, parallelism and pipelining. Calculations that are similar should be done together. For instance, shading calculations are usually similar at all points on a surface, so an entire surface should be shaded at the same time.

- Common Representation: Most of the algorithm should work with a single type of basic geometric object. All geometric primitives are turned into micropolygons[2], which are flat-shaded subpixel-sized quadrilaterals. All of the shading and visibility calculations are performed exclusively on micropolygons.

- Locality: Paging and data thrashing (when the data is too big to fit in memory) should be minimized.

    - Geometric Locality: Calculations for a geometric primitive should be performed without reference to other geometric primitives. Procedural models should be computed only once and should not be kept in their expanded form any longer than necessary.

[2]Nowadays, they are called fragments.

    - Texture Locality: Only the textures currently needed should be in memory, and textures should be read off the disk only once.

- Linearity: The rendering time should grow linearly with the size of the model.
- Large models: There should be no limit to the number of geometric primitives in a model.
- Back door: There should be a back door in the architecture so that other programs can be used to render some of the objects. This give us a very general way to incorporate any new technique (though not necessarily efficiently).
- Texture maps: Texture map access should be efficient, as we expect to use several textures on every surface.

12.2. Algorithm

The REYES algorithm is described in Figure 12.2. Each primitive is read from the model, and for each of them:

1. The primitive is bounded (for instance, with a bounding box).
2. If the primitive is not on screen, it is culled, which means that it is discarded and not processed anymore.
3. If part of the primitive is on screen, then the diceable test is performed. Some primitives can be diced directly (converted into micropolygons), while others need to be split into smaller primitives until they are diceable. If the primitive is not diceable, it is split into smaller primitives, and introduced back into the start of the model processing pipeline.

   It should be noted that a geometric primitive that spans the ε and near planes may cause problems, because the z coordinate is too small and can therefore result in problems regarding the perspective division. Therefore, those primitives are marked as not diceable, and therefore split, until their pieces can be culled or processed.

Once the primitive is diced into micropolygons, the following steps are performed:

1. The micropolygons are shaded. This is done by evaluating the shader at the center of each micropolygon, and then assigning the resulting color to the entire micropolygon. Here is where the textures are accessed, and where the programmable shader is executed.
2. After that, sampling is performed. Each micropolygon is sampled with 16 samples per pixel, avoiding therefore aliasing artifacts.
3. Then, visibility is computed. This is done by projecting the micropolygons onto the screen, and then determining which one is visible at each sample point. This is done with a z-buffer, which stores the depth of the closest micropolygon at each sample point. Here is where the back door is used.

4. Finally, the samples are filtered to produce the final pixel colors, which are stored in the picture.

After all the primitives have been processed, the picture is complete and can be output to a file or displayed on the screen.

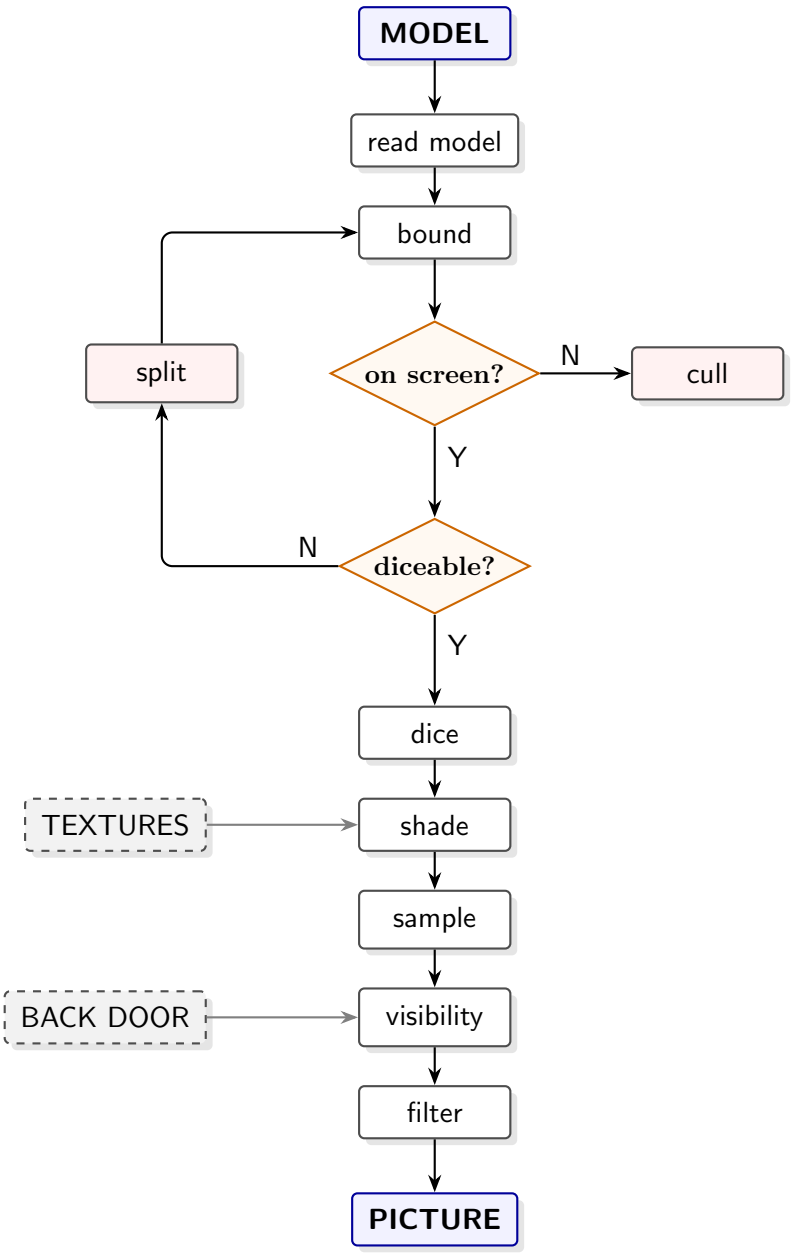

Figura 12.2: REYES algorithm.

13. Image Compression

Image compression is a technique used to reduce the size of image files. For instance, a image encoded using 8 bits for each of the RGB channels (i.e., 24 bits per pixel) in a 3200×2000 pixel image would require 18,31 MB of storage. Applying different types of compression can significantly reduce the file size, making it easier to store and transmit images. There are two main types of image compression: lossless and lossy. Lossless compression allows the original image to be perfectly reconstructed from the compressed data, while lossy compression results in some loss of quality but can achieve much higher compression ratios.

13.1. Lossless Compression

As already mentioned, lossless compression allows the original image to be perfectly reconstructed from the compressed data, resulting in no loss of quality. We will first cover some of the most common lossless compression techniques, and then the main lossless image formats.

Observación. Even though compression techniques are designed to reduce file size, there is no technique that can guarantee that all files will be compressed. If that were the case, we could compress a file repeatedly until we get a file of size zero, which is not possible. In practice, most compression techniques will be able to compress most files, but there will always be some files that cannot be compressed or that may even increase in size after compression.

13.1.1. Run Length Encoding

The Run Length Encoding (RLE) is a simple lossless compression technique that works by replacing sequences of repeated characters with a single character and a count of how many times it is repeated. For example:

$$\texttt{AAAABBBCCDAA} \rightarrow \texttt{4A3B2C1D2A}$$

There are always some conventions that should be previously established to avoid ambiguity. For instance, when a character is repeated only once, we can choose to omit the count and just write the character itself. In that case, the previous example would be encoded as:

$$\texttt{AAAABBBCCDAA} \rightarrow \texttt{4A3B2CD2A}$$

However, some problems may arise:

- If numbers also appear in the original string, we need to establish a convention to distinguish between counts and characters. A control char (X for instance) can be used to indicate that the following character is a count. For example:

  `12121111111122222212111` → `1212X91X6212X31`

  This still has some problems, as the count could be greater than 9, which would require more than one character to represent it. In that case, we can use a different control char (e.g., Y) to indicate the final of the count.

- If the original string contains the control char itself, we need to establish a convention to escape it. For example, we can use a double control char to indicate that the following character is the control char itself. That would therefore require us to use two control chars to represent a single control char in the original string, which could increase the size of the compressed string.

13.1.2. LZ77 Encoding

This technique, called LZ77 because it was developed by Abraham Lempel and Jacob Ziv in 1977, is a lossless compression algorithm that works by converting the input data into a sequence of triples. Two areas are used:

- A sliding window that contains the most recently processed data.
- A look-ahead buffer that contains the next portion of the input data to be processed.

The size of each of the areas is fixed, determined by the implementation of the algorithm, and is usually of a few KB. The algorithm works by finding a match of the start of the look-ahead buffer in the sliding window. It is encoded as a triple that consists of:

- Offset: The position of the start of the match in the sliding window.
- Length: The length of the match.
- Next character: The next character in the look-ahead buffer that follows the match.

Therefore, with each triplet, the match and an additional character are encoded. If there is no match, the offset and length are set to zero, and the next character is the first character in the look-ahead buffer. The sliding window and look-ahead buffer are then updated, and the process continues until all input data has been processed. An example of the LZ77 encoding of the string "ANANAS" with a sliding window of size 8 and a look-ahead buffer of size 4 is shown in Table 13.1.

Regarding the decoding process, the triples are read sequentially, and the original data is reconstructed by using the offset and length to copy the matched string from the sliding window and appending the next character. It should be noted that no look-ahead buffer is needed for decoding, as the original data can be reconstructed solely from the sliding window and the next character. An example of the decoding process is shown in Table 13.2.

Tabla 13.1: LZ77 encoding of the string “ANANAS”.

Sliding Window								Look-Ahead				Output
0	1	2	3	4	5	6	7	0	1	2	3	
								A	N	A	N	(0, 0, ‘A’)
							A	N	A	N	A	(0, 0, ‘N’)
						A	N	A	N	A	S	(6, 2, ‘A’)
			A	N	A	N	A	S				(0, 0, ‘S’)
		A	N	A	N	A	S					Done

Tabla 13.2: LZ77 decoding of the string “ANANAS”.

Output	Sliding Window							
	0	1	2	3	4	5	6	7
(0, 0, ‘A’)								A
(0, 0, ‘N’)							A	N
(6, 2, ‘A’)				A	N	A	N	A
(0, 0, ‘S’)			A	N	A	N	A	S
Done								

As the reader may expect, this algorithm works really good with periodic data (`0101010101...0101`), as it can find long matches in the sliding window. However, it does not work well with data with no repetition (`0123456789ABCDEFGHIJ`), as it cannot find any match in the sliding window, and therefore, the output will be larger than the original data. An aditional problem of this algorithm is that there is not only one way to encode a string, as there may be multiple matches in the sliding window, and the choice of which match to use can affect the final output. This is shown in Table 13.3, where in the fifth row, there are two possible triplets that produce no difference, but in the sixth row, there are two possible triplets whose election changes the final size of the output. Choosing the triplet with the longest match would produce a smaller output, but it requires to go over the whole sliding window each time, which is more computationally expensive.

13.1.3. LZW Encoding

The LZW (Lempel-Ziv-Welch) encoding is a lossless compression algorithm that works by building a dictionary of previously seen sequences of characters. The algorithm starts with an initial dictionary, usually of 12 bits, where the first 8 bits are used to represent the ASCII characters. The process of encoding works as following:

1. The next character from the input data is read and added to the current sequence of characters being processed.

2. There are two possible cases:

Tabla 13.3: LZ77 encoding of `0120101601015` where different options may be chosen.

Sliding Window								Look-Ahead				Output
0	1	2	3	4	5	6	7	0	1	2	3	
								0	1	2	0	(0, 0, ‘0’)
							0	1	2	0	1	(0, 0, ‘1’)
						0	1	2	0	1	0	(0, 0, ‘2’)
					0	1	2	0	1	0	1	(5, 2, ‘0’)
		0	1	2	0	1	0	1	6	0	1	(3, 1, ‘6’) or (6, 1, ‘6’)
0	1	2	0	1	0	1	6	0	1	0	1	(0, 2, ‘0’) or (3, 4, ‘5’)

Tabla 13.4: LZW encoding of the string “LZWLZ7”.

Remaining Input	Current Sequence	In Dict.?	Output	Dict. Update
ZWLZ7	L	Yes	-	-
WLZ7	LZ	No	L	LZ= ⟨256⟩
WLZ7	Z	Yes	-	-
LZ7	ZW	No	Z	ZW= ⟨257⟩
LZ7	W	Yes	-	-
Z7	WL	No	W	WL= ⟨258⟩
Z7	L	Yes	-	-
7	LZ	Yes	-	-
-	⟨256⟩7	No	⟨256⟩	⟨256⟩7= ⟨259⟩
-	7	Yes	7	-

- If the current sequence of characters is not in the dictionary, it is added to the dictionary with the next available code, and the code for the previous sequence of characters is outputted. The current sequence is then reset to the last character read.
- If the current sequence of characters is in the dictionary, we go back to step 1.

An example of the LZW encoding of the string “LZWLZ7” is shown in Table 13.4. The initial dictionary contains the ASCII characters, and the next available code is 256.

A positive aspect that LZW has is that the dictionary is not needed in order to decode, as the dictionary is built during the decoding process as well. When a code is read during the decoding process, the corresponding sequence of characters is outputted (the starting dictionary is the same as in the encoding process), and a new entry is added to the dictionary that consists of the previous code followed by the first character of the current code. An example of the decoding process is shown in Table 13.5.

A positive aspect of this algorithm is that it is deterministic, as there is only one

Tabla 13.5: LZW decoding of the string "LZW⟨256⟩7".

Input Code	Output	Dict. Update
L	L	-
Z	Z	LZ= ⟨256⟩
W	W	ZW= ⟨257⟩
⟨256⟩	LZ	WL= ⟨258⟩
7	7	⟨256⟩7= ⟨259⟩

way to encode a string. However, the dictionary will eventually overflow. There are some approaches to this problem:

- Reset: When the dictionary is full, it can be reset to the initial state, which would allow the algorithm to continue encoding new data, but it would also lose all the previously stored sequences in the dictionary, which could lead to larger output if those sequences are needed again.
- Freeze: When the dictionary is full, it can be frozen, which means that no new entries will be added to the dictionary, but the existing entries can still be used for encoding. This approach would allow the algorithm to continue encoding new data without losing any previously stored sequences, but it would also limit the compression ratio that can be achieved.

Lastly, a worst-case scenario for this algorithm is when the input data consists of a sequence of characters that are not repeated, as each character would use 12 (or the dictionary size) bits insted of 8 bits, which would result in a larger output than the original data.

13.1.4. Huffman Encoding

Huffman encoding is a lossless compression algorithm that works by assigning variable-length codes to input characters, with shorter codes assigned to more frequent characters. A set of symbols and their weigths (usually probabilities) is required. Using that set, a binary code is assigned to each symbol taking into account that characters with higher weights must have shorter codes. However, given that the codes are variable-length, in a sequence of 0s and 1s, it is not possible to determine where one code ends and the next one begins.

- If $A = 10$, $B = 01$ and $C = 0$, the sequence 10010 could be decoded as ABC or ACA.

To solve this problem, the codes must be prefix-free, which means that no code can be a prefix of another code. This way, it is always possible to determine where are the boundaries between codes.

In the following subsection we will see how to construct the prefix-free codes from the set of symbols and their weights, but let's see first how is the encoding and decoding process.

- Encoding: The input data is read character by character, and the corresponding code for each character is outputted.

- Decoding: The sequence of 0s and 1s is read bit by bit, and the corresponding character is outputted when a valid code is recognized, using the prefix-free property to determine the boundaries between codes.

Therefore, this process is straightforward. In order to decode the data, the symbols and their corresponding weights (or directly the codes) must also be saved, so this will slightly increase the size of the output, but it is necessary for the decoding process.

Constructing the Prefix-Free Codes

In order to construct the prefix-free codes from the set of symbols and their weights, a binary tree called *the Huffman Tree* is used. The process is as follows:

1. Each symbol is represented as a leaf node in the tree, and the weight of each symbol is assigned to the corresponding leaf node.

2. All of the nodes are stored in a priority queue, where the node with the smallest weight has the highest priority.

3. While there is more than one node in the queue:

   a) The two nodes with the smallest weights (they are in the front of the queue) are removed from the queue.

   b) A new node is created with these two nodes as children, and the weight of the new node is the sum of the weights of the two child nodes.

   c) The new node is added back to the priority queue.

4. The number of nodes will decrease by one in each iteration, and at the end, there will be only one node left in the queue, which will be the root of the tree.

Once the Huffman Tree is constructed, the codes for each symbol can be generated by traversing the tree from the root to the corresponding leaf node. A common convention is to assign a 0 for a left edge and a 1 for a right edge, so the code for each symbol will be the sequence of 0s and 1s that are encountered along the path from the root to the leaf node.

- Codes are prefix-free because all the symbols are represented as leaf nodes. In order to have a prefix code, one symbol would have to be child of another symbol, which is not possible.

- The symbols with higher weights will have shorter codes because they will be closer to the root of the tree, while the symbols with lower weights will have longer codes because they will be farther from the root of the tree.

Huffman encoding is optimal in the sense that it produces the shortest possible average code length for a given set of symbols and their weights. However, it requires the knowledge of the symbol frequencies (or weights) in advance, which may not always be possible. In such cases, adaptive Huffman coding can be used, which builds the Huffman Tree dynamically as the input data is processed.

13.1.5. Deflate Encoding

Deflate is a lossless compression algorithm that combines the LZ77 and Huffman encoding techniques. It works by first applying the LZ77 encoding to the input data, which produces a sequence of triples. Then, the output of the LZ77 encoding is further compressed using Huffman encoding, which assigns variable-length codes to the different numbers and characters that appear in the LZ77 output. The resulting compressed data is a sequence of bits that can be efficiently stored and transmitted. Deflate is widely used in various file formats, such as ZIP and PNG, due to its good compression ratio and fast decompression speed.

13.2. Image Formats

There are several image formats that use the aforementioned compression techniques to reduce the file size. Some of the most common lossless image formats include:

- BMP/TGA: These formats can be used with or without compression. When compression is used, they typically use RLE encoding, where images are stored as a sequence of runs of pixels with the same color, where each pixel is represented by a fixed number of bits (usually 24 or 32 bits per pixel).

- GIF: This format uses LZW encoding and only supports a maximum of 256 colors. A few GIF versions exist, some of them allowing a single transparent color and stacking multiple images together to generate an animation.

- PNG: Initially developed to solve some patent issues with GIF, this format uses Deflate encoding and supports a wide range of colors (up to 16 bits per channel) and transparency (8 or 16 bits for the alpha channel). It is supported by most web browsers but does not support animation.

  In addition, the PNG format has a previous step before applying the Deflate encoding, which is called *prefiltering*. This step just prepares the image data for the Deflate encoding by applying a simple transformation to the pixel values, which can help to create more repetitive patterns in the data, which can be more efficiently compressed by the Deflate algorithm. The prefiltering step can be avoided by setting the filter type to None. In the other hand, the value stored in the pixel is the difference between the original pixel value and the value predicted by the filter, which depends on the filter type:

  - Sub: The predicted value is the pixel value to the left of the current pixel. This filter is effective for images with horizontal patterns or gradients.

- Up: The predicted value is the pixel value above the current pixel.
  This filter is effective for images with vertical patterns or gradients.
- Average: The predicted value is the average of the pixel values to the left and above of the current pixel.
  This filter is effective for images with diagonal patterns or gradients.
- Paeth: Three pixels are considered: the pixel to the left L, the pixel above U and the pixel to the upper left UL. A value $P = L + U - UL$ is calculated, and the predicted value is the one among L, U and UL that is closest to P.
  This filter is effective for images with complex patterns or gradients, as it detects the direction of the gradient and chooses the most appropriate predictor accordingly.

This format is also lossless as the original pixel values can be perfectly reconstructed from the filtered values and the filter type used. Given that the image is reconstructed from left to right and from top to bottom, the original pixel value can be calculated by adding the filtered value to the predicted value, which can be calculated using the previously reconstructed pixels.

13.2.1. Lossy Compression: JPEG

Joint Photographic Experts Group (JPEG) is a widely used lossy image compression format with adjustable compression ratios (can also be $1:1$ for lossless compression). It works for all types of static images, with no restrictions on the color depth. In the following sections, the different steps of the process will be adressed.

Color Space conversion to YUV

YUV is a color space that separates the luminance (Y) from the chrominance (U and V) components of an image. Each of the components represents a different aspect of the image:

- Y (luminance): This component represents the brightness of the image.
- U chrominance: This component represents the color change from yellow to blue.
- V chrominance: This component represents the color change from green to red.

There are mathmatical formulas (matrix transformations) that can be used to convert the RGB color space to the YUV color space and vice versa.

Block based color sub sampling

This step is based on the fact that the human eye is more sensitive to changes in brightness (luminance) than to changes in color (chrominance). Therefore, the chrominance components can be subsampled without significantly affecting the perceived quality of the image. The image is divided into blocks of 8×2 pixels, and each of the chrominance components (U and V) is subsampled using the $J : a : b$ notation, where:

- J is the horizontal sampling reference, which is usually set to 4.
- a is the number of chrominance samples in the first row of the block (out of J samples).
- b is the number of chrominance samples in the second row of the block (out of J samples).

The subsampling is usually done by averaging the chrominance values of the pixels in the block. Some common subsampling formats include:

- 4:4:4: No subsampling, all chrominance samples are retained.
- 4:2:2: For each block of 8×2 pixels, in the first row 4 chrominance samples are retained and in the second row 4 chrominance samples are retained, resulting in half the number of chrominance samples compared to the luminance samples.
- 4:2:0: For each block of 8×2 pixels, in the first row 4 chrominance samples are retained and in the second row 0 chrominance samples are retained, resulting in a quarter of the number of chrominance samples compared to the luminance samples.

This is a lossy step, as the original chrominance values cannot be perfectly reconstructed from the subsampled values, but it can significantly reduce the file size without significantly affecting the perceived quality of the image.

Block Division

In the following steps, the image is processed in blocks of 8×8 pixels. This is done to take advantage of the spatial redundancy in the image, as pixels that are close to each other are likely to have similar values.

In addition, each of the different components (Y, U and V) is processed separately, and an index shift is applied to the pixel values to center them around zero. For instance, if the pixel values are in the range $[0, 255]$, a shift of -128 is applied to center them around zero, resulting in a new range of $[-128, 127]$.

Discrete Cosine-transformation

The Discrete Cosine Transform (DCT) is a mathematical transformation that converts a signal from the spatial domain to the frequency domain. DCT uses a set of basis functions that are cosine waves of different frequencies. Given a value of frecuency t, $DCT(t)$ represents the contribution of that frequency to the original signal, which is calculated using the Equation 13.1.

$$DCT(t) = C(t) \cdot \sqrt{\frac{2}{N}} \cdot \sum_{x=0}^{N-1} f(x) \cdot \cos\left(\frac{(2x+1) \cdot t\pi}{2N}\right) \tag{13.1}$$

where:

- N is the number of samples in the original signal.

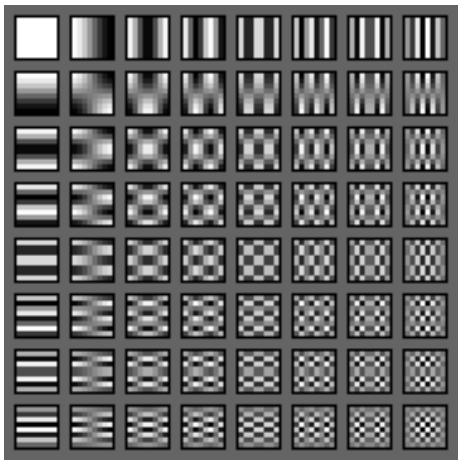

Figura 13.1: Visualization of the DCT basis functions for an 8×8 block.

- $f(x)$ is the value of the original signal at position x (in this case, the pixel values in the block).
- $C(t)$ is a normalization factor that is defined as:

$$C(t) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } t = 0 \\ 1 & \text{otherwise} \end{cases}$$

Regarding JPEG compression, the DCT is applied to each block of 8×8 pixels using also 64 basis functions represented in Figure 13.1. $DCT(i, j)$ represents the contribution of the frequency component with horizontal frequency i and vertical frequency j to the original block. Therefore:

- Increasing the value of i corresponds to increasing the horizontal frequency, resulting in more precise representation of horizontal details/patterns in the image.
- Increasing the value of j corresponds to increasing the vertical frequency, resulting in more precise representation of vertical details/patterns in the image.
- The $DCT(0, 0)$ component, also known as the DC coefficient, represents the average value of the block.
- If n is the maximum value of i and j, the $DCT(n, n)$ component represents the highest frequency component in the block, which corresponds to the finest details in the image.

This idea is shown in Figure 13.1, where the DCT basis functions for an 8×8 block are visualized. The value of $DCT(i, j)$ represents the contribution of the corresponding basis function to the original block, and is calculated using the Equation 13.2, which is a two-dimensional extension of the DCT:

$$DCT(i,j) = C(i) \cdot C(j) \cdot \frac{1}{\sqrt{2N}} \cdot \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x,y) \cos\left(\frac{(2x+1) \cdot i\pi}{2N}\right) \cos\left(\frac{(2y+1) \cdot j\pi}{2N}\right) \tag{13.2}$$

where:

- N is the number of samples in each dimension of the original block (in this case, 8).
- $f(x, y)$ is the value of the original block at position (x, y) (the pixel value at that position).
- $C(i)$ and $C(j)$ are normalization factors defined as:

$$C(t) = \begin{cases} \frac{1}{\sqrt{2}} & \text{if } t = 0 \\ 1 & \text{otherwise} \end{cases}$$

This way, the original block is now represented in the frequency domain, where also 64 coefficients are obtained but have different values and represent different aspects.

Quantization of the DCT Coefficients

In this step, a quantization matrix is used to reduce the precision of the DCT coefficients, which results in a loss of information but also in a significant reduction of the file size (as we will see). A matrix with the same dimension as the DCT coefficients (in this case, 8×8) is used, where each element of the matrix represents the quantization step for the corresponding DCT coefficient.

- Elements closer to the top-left corner of the matrix (lower frequencies) usually have smaller quantization steps (values in the matrix), as they represent important general information about the image that should not be lost.
- Elements closer to the bottom-right corner of the matrix (higher frequencies) usually have larger quantization steps, as they represent finer details in the image that can be more easily discarded without significantly affecting the perceived quality of the image. Therefore, bigger quantization steps are used to take those values to 0.

This way, the 8×8 block of DCT coefficients is transformed into a new 8×8 block, where the value stored in the position (i, j) is:

$$\overline{DCT}(i, j) = \text{round}\left(\frac{DCT(i, j)}{Q(i, j)}\right)$$

where $Q(i, j)$ is the quantization step for the coefficient at position (i, j), and round$(\cdot)$ is the rounding function that rounds the result to the nearest integer.

This step is also lossy given two reasons:

- The original DCT coefficients cannot be perfectly reconstructed from the quantized coefficients, as the quantization process involves rounding, which can lead to a loss of precision.

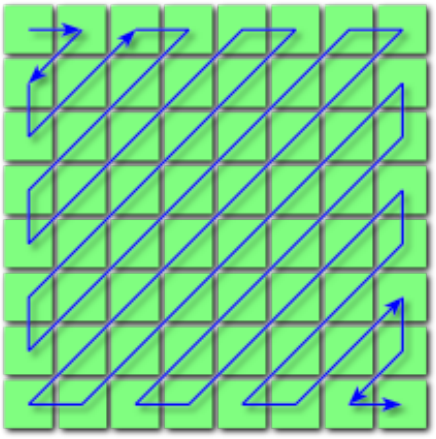

Figura 13.2: Zig-zag order for an 8×8 block.

- The quantization process can lead to a significant loss of information where the quantized coefficients are taken to 0, which can result in a significant loss of details in the image, especially in areas with high frequencies (fine details).

The election of the matrix Q is crucial, as it can significantly affect the compression ratio and the perceived quality of the image. Some softwares allow the user to specify a quality factor, which in fact implies using different matrixes.

Coefficient serialization

Once the quantized DCT coefficients are obtained, they need to be serialized in order to be further compressed using Huffman encoding. This process is called "dove tailing" or "zig-zag scanning", because the coefficients are serialized in a zig-zag order, starting from the top-left corner of the block and moving towards the bottom-right corner. This way, the coefficients are ordered from lower frequencies to higher frequencies, which allows to take advantage of the fact that higher frequency coefficients are more likely to be zero after quantization, which can lead to better compression ratios when using Huffman encoding. The zig-zag order for an 8×8 block is shown in Figure 13.2.

In addition, the serialized coefficients are not stored directly, but they are stored as a difference between the current coefficient and the previous one, which leads to more repetitive patterns in the data, which can be more efficiently compressed using Huffman encoding.

Coefficient encoding